

postgres存储管理概况

一、内存管理

1.1、缓冲池管理

1.1.1、共享缓冲区

1.1.2、本地缓冲区

1.2、Cache机制

1.2.1、系统表元组缓存

(1) SysCache 初始化

(2) CatCache 中缓存元组的组织

(3)在CatCache中查找元组

a)精确匹配

b)部分匹配

1.2.2、表基本信息缓存

(1)插入操作

(2)查找查找

(3)删除操作

1.2.3、Cache同步

1.3、空闲空间管理

1.3.1、VACUUM机制

1.3.1.1、GUC 参数对VACUUM行为的影响（补充）

1.3.2、Lazy VACUUM

1.3.3、Full VACUUM

1.4、进程间通信机制管理/IPC(进程间通信)

1.4.1、共享内存管理

1.4.2、SI Message

1.5、内存上下文（Memory Context）

1.5.1、MemoryContext

1.5.2、内存上下文初始化与创建

1.5.3、内存上下文中内存的分配

1.5.4、内存上下文释放

二、外存管理

2.1、表和元组组织方式

2.1.1、堆文件

- a) 普通堆 (ordinary cataloged heap)
- b) 临时堆 (temporary heap)
- c) 序列 (SEQUENCE relation, 一种特殊的 单行表)
- d) TOAST 表 (TOAST table)

2.1.2、文件存储形式

2.2、表文件管理

2.2.1、分页存储管理

2.2.2、行式存储

2.2.3、系统表

2.3、磁盘管理器

2.3.1、SMGR实现

2.3.2、VFD机制

2.3.3、LRU池

2.4、空间管理

2.4.4、空闲表空间映射

2.4.5、可见性映射表

VM 文件结构

接口函数

2.5、大的数据存储管理

2.5.1、TOAST表结构

2.5.2、TOAST操作

2.5.3、大对象

三、表操作和元组操作

3.1、表操作

3.1.1、打开表

3.1.2、扫描表

3.1.3、同步扫描

3.2、元组操作

3.1.1、插入元组

3.1.1.1、heap_form_tuple 函数

3.1.1.2、heap_fill_tuple 函数

3.1.1.3、heap_insert 函数

3.1.1.4、RelationPutHeapTuple 函数

3.1.2、删除元组

3.1.2.1、heap_delete 函数

3.1.3、更新元组

3.1.3.1、heap_update 函数

四、ResourceOwner资源跟踪

4.1、ResourceOwner结构和功能

4.2、ResourceArray

4.3、Portal 的 ResourceOwner

4.4、基本API概况

五、实验部分

5.1、FULL VACUUM实验

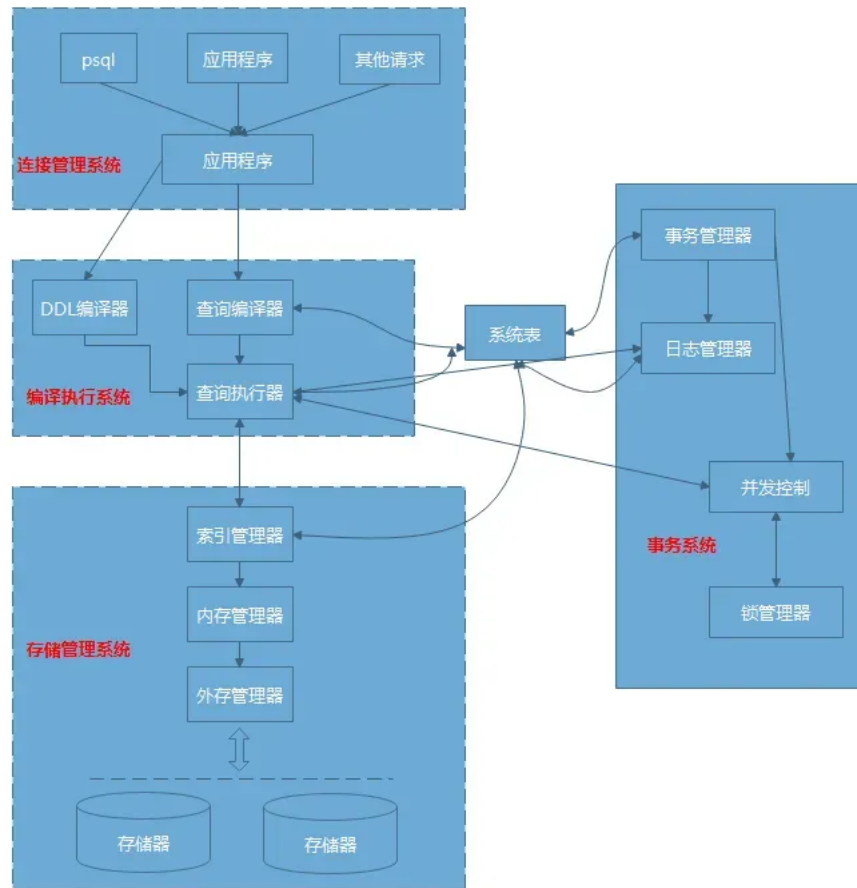
5.2、扫描表

5.3、TOAST

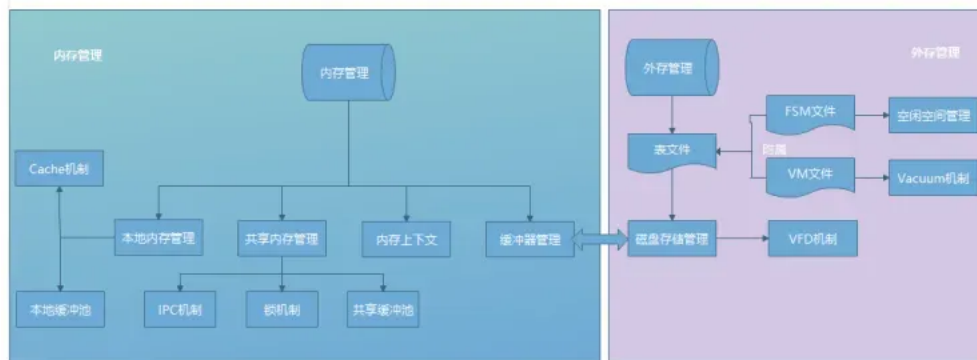
5.4、系统表

数据库管理系统的任务本质上是向存储设备上写入数据或者从存储设备上读出数据，因此对于一个DBMS来说，存储的管理是一项非常基础和重要的技术。

存储管理器提供了一组统一的管理外存与内存资源的功能模块，所有对外存与内存的操作都交由存储管理器处理，可以认为存储管理器是数据库管理系统与物理存储设备的接口。PostgreSQL体系结构图如下图所示，本篇主要介绍PostgreSQL的存储管理相关内容

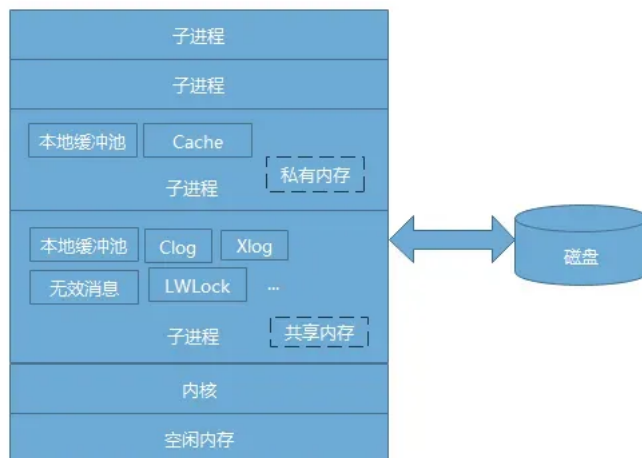


在PostgreSQL中，有专门的模块负责管理存储设备（包括内存和外存），我们称之为存储管理器。



一、内存管理

PostgreSQL内存管理包括共享内存和本地内存，PostgreSQL内存管理如下图所示：



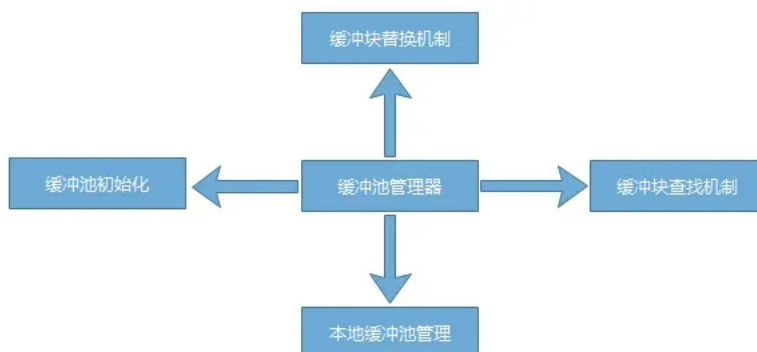
1.1、缓冲池管理

如果需要访问的系统表元组在 Cache 中无法找到或者需要访问普通表的元组，就需要对缓冲池进行访问。

任何对于表、元组、索引表等的操作都在缓冲池中进行，缓冲池的数据调度都以磁盘块为单位，需要访问的数据以磁盘块为单位调用函数smgrread写入缓冲池，而smgrwrite将缓冲池数据写回到磁盘。

调入缓冲池中的磁盘块称为缓冲区、缓冲块或页面，多个缓冲区组成缓冲池。

PostgreSQL有两种缓冲池：共享缓冲池和本地缓冲池。共享缓冲池作普通可共享表的操作场所；本地缓冲池则用作仅本地可见的临时表的操作场所。对缓冲池中缓冲区的管理通过两种机制完成：pinlock。pin对缓冲区的访问计数器。lock机制为缓冲区的并发访问提供了保障，当有进程对缓冲区进行写操作时加EXCLUSIVE锁，读才做时加SHARE锁，其意义和数据库的锁机制是类似的。缓冲池模型图如下图所示：



1.1.1、共享缓冲区

存储着所有进程的公共数据，例如锁变量、进程通信状态、缓冲区等

1.1.2、本地缓冲区

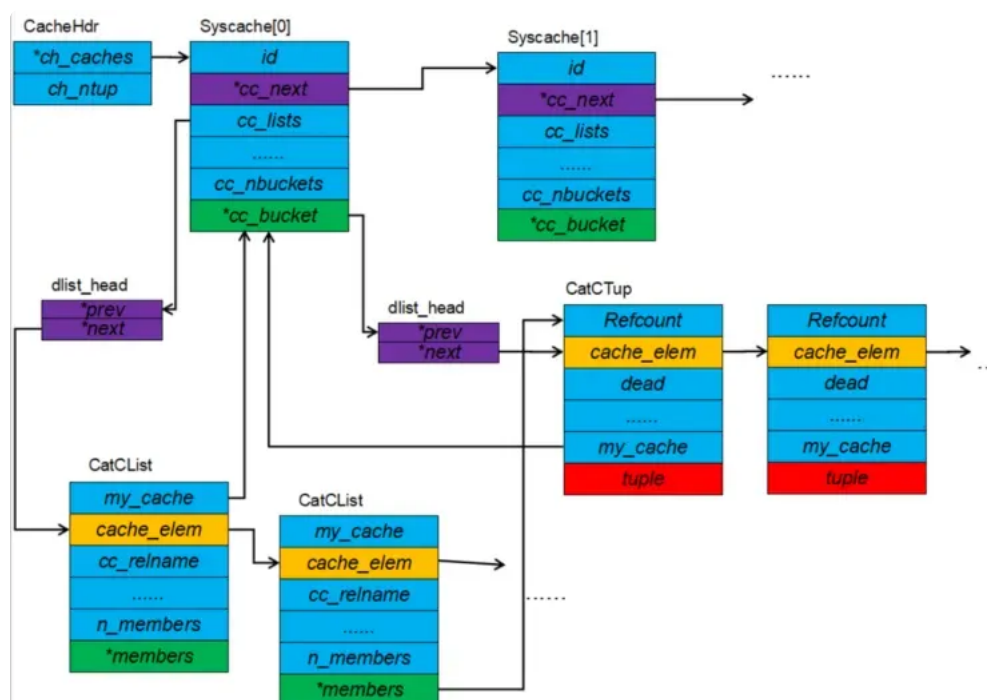
存储着属于该进程的Cache（高速缓存）、事务管理信息、进程信息等。

1.2、Cache机制

为了减少对多个系统表的修改。PostgreSQL 在每个进程的本地内存区域内设立了两种缓存。

1.2.1、系统表元组缓存

SysCache 主要用来缓存系统表元组。从实现上看 SysCache 就是一个数组，数组的长度为预定义的系统表的个数。



上图是SysCache的内存结构图。在 PostgreSQL 中，SysCache是一种高效的机制，设计用来缓存数据库中系统表的元组数据。

这些系统表包括 PostgreSQL 数据库内部使用的各种元数据，如用户权限、数据类型、函数定义等。SysCache 在实现上由一个预定义长度的数组构成，每个数组元素对应一个特定的系统表，并且每个元素具体关联一个 CatCache 结构。

每个 **CatCache** 结构通过使用哈希表来存储和管理其缓存的元组。哈希存储机制允许 CatCache 快速通过关键字定位到特定的元组，这些关键字通常是查询系统表时使用的主要字段。例如，如果系统表经常因为某个特定的标识符或名称字段被访问，CatCache 会针对这些字段构建哈希索引，确保即使在大量访问请求的情况下，元组的检索仍然能保持高效和快速。

这种策略显著提升了对系统元数据访问的性能，支持 PostgreSQL 数据库在解析查询、验证用户权限等操作中的高效率 and 响应速度。

(1) SysCache 初始化

在对Postgres进程初始化时，会对SysCache进行初始化，将查找系统表元组的关键信息写入到CatCache数组的元素中。涉及到的数据结构如下：

- cacheinfo：存储所有系统表的CatCache描述信息，其中CatCache信息的数据类型为cachedesc。

```
1 struct cachedesc
2 {
3     Oid      reloid;      /* OID of the relation being cached */
4     Oid      indoid;     /* OID of index relation for this cache */
5     int      reloidattr; /* attr number of rel OID reference, or 0 */
6     int      nkeys;      /* # of keys needed for cache lookup */
7     int      key[4];     /* attribute numbers of key attrs */
8     int      nbuckets;   /* number of hash buckets for this cache */
9 };
10
11 static const struct cachedesc cacheinfo[] = {
12     {AggregateRelationId, /* AGGFNOID */
13      AggregateFnoIdIndexId,
14      1,
15      {
16          Anum_pg_aggregate_aggfnoid,
17          0,
18          0,
19          0
20      },
21      16
22     },
23     ...
24 }
25 }
```

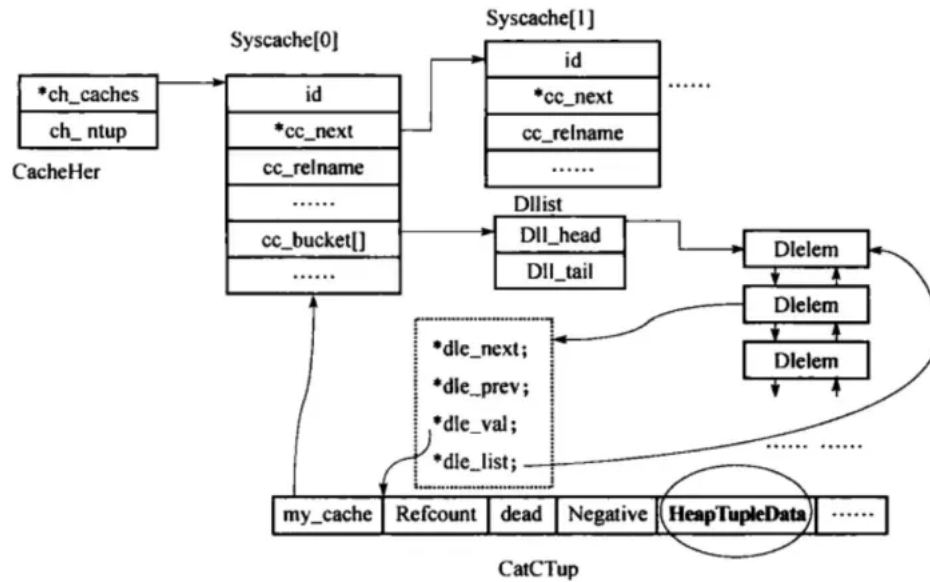
- catcacheheader：CatCache使用cc_next字段构成一个单向链表，头部使用全局变量CacheHdr记录

```
1 typedef struct catcacheheader
2 {
3     slist_head ch_caches; /* head of list of CatCache structs */
4     int        ch_ntup;   /* # of tuples in all caches */
5 } CatCacheHeader;
```

同时，InitCatalogCache函数将被调用，它在 PostgreSQL 中负责初始化系统缓存（SysCache），它为每个系统表元组建立并配置缓存结构。

这个过程包括为 SysCache 分配基本信息结构、设置内存上下文，并构建用于快速元组检索的哈希表。紧接着，InitCatalogCachePhase2 函数进一步完善这些缓存结构，创建必要的索引，这些索引优化了基于关键系统表列的查询性能，使得元组的访问变得更加高效。这两个步骤共同确保了 PostgreSQL 在处理数据库元数据时的性能和响应速度。

(2) CatCache 中缓存元组的组织



以上是CatCache中缓存元组的组织：

CatCache中的cc_bucket是一个可变数组。cc_bucket数组中的每一个元素都表示一个Hash桶，元组的键值通过Hash函数可以映射到cc_bucket数组的下标。每一个Hash桶都被组织成一个双向链表(Dlist)，其中的节点为Dlist_node类型。

```
1 typedef struct dlist_node dlist_node;
2 struct dlist_node
3 {
4     dlist_node *prev;
5     dlist_node *next;
6 };
```

具有同一个hash值得元组被缓存在同一个hash桶中，每一个hash桶中的缓存元组都被先包装成Dlist_node结构并链接成一个链表。因此在查找某一个元组时，需要先计算其Hash键值并通过键值找到其所在的Hash桶，之后要遍历Hash桶的链表逐一比对缓存元组。为了尽最减少遍历Hash桶的代价，在组织Hash桶中链表时，会将这一次命中的缓存元组移动到链表的头部，这样下一次查找同一个元组时可以在尽可能少的时间内命中。

CatCache中的缓存元组将先被包装成CatCTup形式，然后加入到其所在Hash桶的链表中。

```
1 typedef struct catctup
2 {
3     int         ct_magic;        /* for identifying CatCTup entries */
4     #define CT_MAGIC 0x57261502
5     uint32      hash_value;      /* hash value for this tuple's keys */
6     Datum       keys[CATCACHE_MAXKEYS];
7     dlist_node  cache_elem;      /* list member of per-bucket list */
8     int         refcount;        /* number of active references */
9     bool        dead;           /* dead but not yet removed? 标记删除*/
10    bool        negative;        /* negative cache entry? 表示实际并不存在的元组*/
11    HeapTupleData tuple;         /* tuple management header */
12    struct catclist *c_list;     /* containing CatCList, or NULL if none */
13
14    CatCache     *my_cache;       /* link to owning catcache */
15 } CatCTup;
```

在CatCTup中通过my_cache和cache_elem分别指向该缓存元组所在的CatCache及Hash桶链表中的节点。一个被标记为“死亡”的CalCTup(dead字段为真)并不会实际从CatCache中删除，但是在后续的查找中它不会被返回。“死亡”的缓存元组将一直被保留在CatCache中，直到没有人访问它，即其refcount变为0。但如果“死亡”元组同时也属于一个CatCList,则必须等到CatCList和CatCTup的refcount都变为0时才能将其从CatCache中清除。CatCTup的negative字段表明该缓存元组是否为一个“负元组”，所谓负元组就是实际并不存在于系统表中，但是其键值曾经用于在CatCache中进行查找的元组。负元组只有键值，其他属性均为空。负元组的存在是为了避免反复到物理表中去查找不存在的元组所带来的I/O开销。

(3)在CatCache中查找元组

在CatCache中查找元组有两种方式：精确匹配和部分匹配。前者用于给定CatCache所需的所有键值，并返回CatCache中能完全匹配这个键值的元组；而后者只需要给出部分键值，并将部分匹配的元组以一个CatCList的方式返回。

a)精确匹配

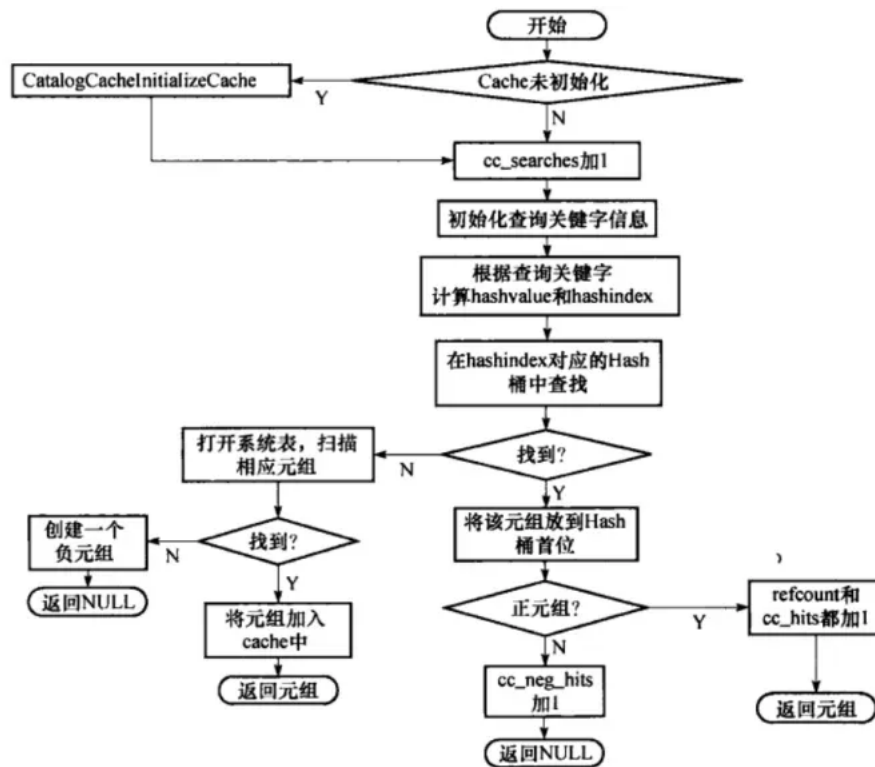
```
1 HeapTuple
2 SearchCatCache(CatCache *cache,
3               Datum v1,
4               Datum v2,
5               Datum v3,
6               Datum v4);
```

SearchCatCache 是 PostgreSQL 中用于在指定的 CatCache 中查找系统表元组的函数。它能使用最多四个键值进行精确匹配查询，这些键值对应于 CatCache 数据结构中定义的搜索键。当查找元组时，首先需要确定要在哪个 CatCache 中进行搜索，这通常通过遍历 SysCache 中的 CatCache 结构体来完成，基于系统表的名字或 OID 定位到具体的 CatCache。

在查找过程中，SearchCatCache 会先对键值进行哈希处理，通过哈希值定位到 cc_bucket 数组中的哈希桶。然后遍历这个桶中的链表，找到满足条件的 CatCTup 元素，同时为了优化性能，将访问过的元素移动到链表头部，并增加命中计数器 cc_hits。

如果在哈希桶中没有找到元素，SearchCatCache 需要扫描物理系统表来确定这个元组是否确实不存在，或者它之前只是没有被缓存。如果找到了元组，它将被加入到哈希桶中；如果没找到，则创建一个“负元组”加入到哈希桶中，这样可以避免以后的不必要的物理扫描，因为此后查找同样的不存在元组时，可以直接从“负元组”得知它并不存在于系统表中。

使用 SearchCatCache 查找元组时，调用者必须注意不能修改返回的元组，并且在使用完毕后需要调用 ReleaseCatCache 函数来释放资源。这个过程提高了查询效率，减少了对物理系统表的直接访问，优化了数据库的性能。具体的流程图如下：



b)部分匹配

```

1 SearchCatCacheList(CatCache *cache,
2                     int nkeys,
3                     Datum v1,
4                     Datum v2,
5                     Datum v3)

```

SearchCatCacheList 是 PostgreSQL 中用于执行部分匹配搜索的函数，它利用 CatCList 结构来存储和管理搜索结果。CatCList 是一种特殊的结构，用于缓存在一个 CatCache 中找到的一组元组。每个 CatCList 包含了多个与搜索条件相匹配的元组（通过 members 数组持有），并记录了实际找到的元组数量（n_members 字段）。

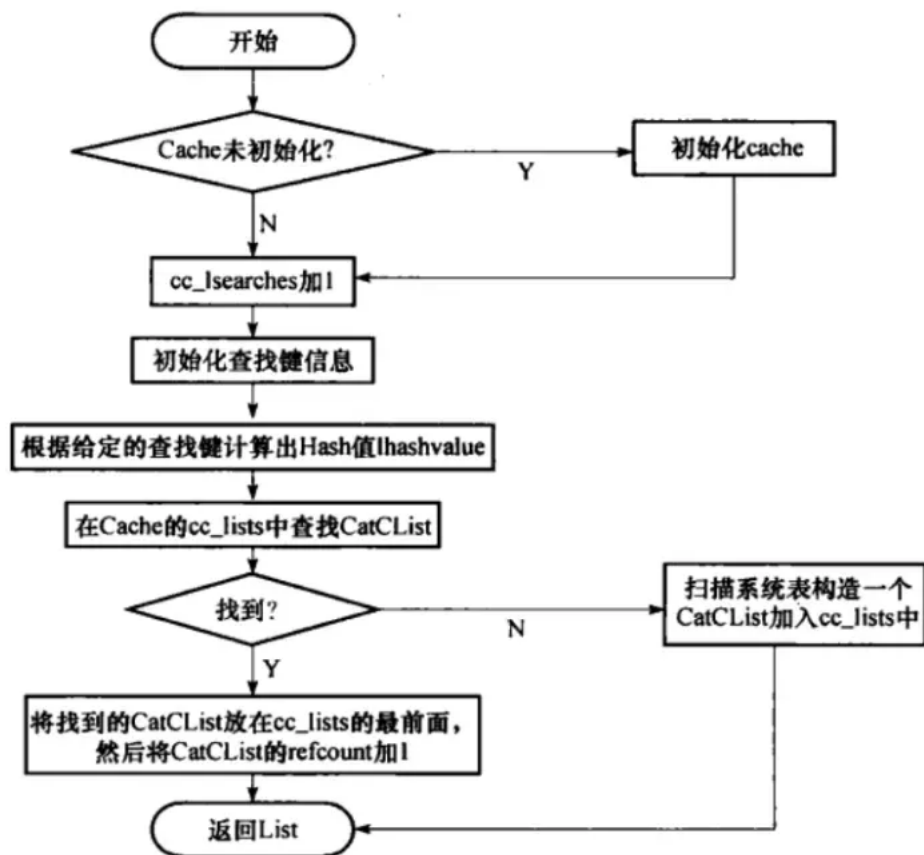
```

1 typedef struct catclist
2 {
3     int      cl_magic;      /* for identifying CatCList entries */
4     #define CL_MAGIC  0x52765103
5
6     uint32    hash_value;    /* hash value for lookup keys */
7
8     dlist_node cache_elem;    /* list member of per-catcache list */
9
10    /*
11     * Lookup keys for the entry, with the first nkeys elements being valid.
12     * All by-reference are separately allocated.
13     */
14    Datum      keys[CATCACHE_MAXKEYS];
15
16    int        refcount;      /* number of active references */
17    bool       dead;          /* dead but not yet removed? */
18    bool       ordered;       /* members listed in index order? */
19    short      nkeys;         /* number of lookup keys specified */
20    int        n_members;     /* number of member tuples */
21    CatCache   *my_cache;      /* link to owning catcache */
22    CatCTup    *members[FLEXIBLE_ARRAY_MEMBER]; /* members */
23 } CatCList;
24

```

当执行部分匹配搜索时，SearchCatCacheList 首先计算搜索键的哈希值，然后尝试在 CatCache 的 cc_lists 链表中查找是否已经有缓存了对应的搜索结果。这一查找过程基于 CatCList 中 tuple 字段的哈希值与搜索键进行对比。如果找到相应的 CatCList，将会更新缓存的命中计数器 cc_hits，将该 CatCList 移动到链表头部，并返回它。若在 CatCache 中没有找到对应的 CatCList，则会扫描物理系统表，构建新的 CatCList 并将其加入到链表头部。

调用 SearchCatCacheList 的函数需要确保不修改返回的 CatCList 对象或其中的元组，并且在使用完毕后，应调用 ReleaseCatCacheList 释放引用。这个机制优化了对系统表部分匹配搜索的处理，提高了查询效率并减少了不必要的系统表访问。具体的流程图如下：



1.2.2、表基本信息缓存

对 RelCache 的管理比 SysCache 要简单许多，原因在于大多数时候 RelCache 中存储的 RelationData 的结构是不变的，因此 PostgreSQL 仅用一个 Hash 表来维持这样一个结构。

RelCache 存放的不是元组，而是 RelationData 数据结构，每一个 RelationData 结构表示一个表的模式信息。其结构如下：

```

1 typedef struct RelationData
2 {
3     RelFileNode rd_node;      /* relation physical identifier */
4     struct SMgrRelationData *rd_smgr; /* 表的文件句柄 */
5     ...
6     Form_pg_class rd_rel;     /* 表在pg_class系统表中对应的元组里的信息 */
7     TupleDesc rd_att;         /* 表的元组描述符，描述了表的各个属性 */
8     Oid rd_id;                /* relation's object id */
9     List *rd_indexlist;       /* list of OIDs of indexes on relation */
10    Bitmapset *rd_indexattr;   /* identifies columns used in indexes */
11    Oid rd_oidindex;           /* OID of unique index on OID, if any */
12    ...
13    Form_pg_index rd_index;    /* pg_index tuple describing this index */
14    ...
15 } RelationData;
  
```

RelCache 的管理相对于 SysCache 来说简单很多，因为 RelationData 结构通常是固定不变的。因此，PostgreSQL 只使用一个以表的 OID 为键的哈希表来维护 RelCache。对于 RelCache 的查找、插入、删除和修改操作都相对直接。

RelCache 的初始化在 InitPostgres 函数中进行，并分为两个阶段：RelationCacheInitialize 和 RelationCacheInitializePhase2。第一阶段初始化创建进程级的哈希表，使用 oid_hash 函数设置哈希键。第二阶段将重要的系统表和索引的模式信息加入 RelCache，如有必要，会利用 pg_internal.init 文件来加速这个过程。

一旦 RelCache 初始化完成，它就可以用来查找表的模式信息。主要操作包括：

(1)插入操作

使用宏 RelationCacheInsert 来将新打开的表的 RelationData 插入到 RelCache。这是通过在哈希表中定位相应的位置并将 RelationData 分配给 reldesc 字段来完成的。

```
typedef struct relidCacheent
{
    Oid      relid;          //表的 OID
    Relation reldesc;        //表的 RelationData 结构指针
}RelIdCacheEnt;
```

(2)查找查找

宏 RelationIdCacheLookup 通过 hash_search 查找特定 OID 的 RelIdCacheEnt 条目，如果找到，则将 reldesc 字段的值赋予相关变量。

```
1 relation_open
2     RelationIdGetRelation
3     RelationIdCacheLookup(relationId, rd);
4     RelationBuildDesc -- no reldesc in the cache, RelationBuildDesc() build one and add it.
5     RelationBuildTupleDesc
6     ....
7     RelationCacheInsert(relation);
```

(3)删除操作

使用宏 RelationCacheDelete 来从哈希表中移除条目，通过 hash_search 并指定 HASH_REMOVE 模式来完成删除。

```
1 RelationClearRelation
2     RelationCacheDelete
```

哈希表在 RelCache 中充当索引的角色，加速了对 RelCache 的查找过程，从而优化了数据库的性能，特别是在频繁访问表的结构信息时。这些操作确保了表的 RelationData 可以被高效地检索和管理。

1.2.3、Cache同步

在PostgreSQL中，每个进程都有属于自己的Cache。换句话说，同一个系统表在不同的进程中都有对应的Cache来缓存它的元组（对于RelCache来说缓存的是一个RelationData结构）。同一个系统表的

元组可能同时被多个进程的Cache所缓存，当其中某个Cache中的一个元组被删除或更新时，需要通知其他进程对其Cache进行同步。

在PostgreSQL的实现中，会记录下已被删除的无效元组，并通过SI Message方式（即共享消息队列方式）在进程之间传递这一消息。收到无效消息的进程将同步地把无效元组（或RelationData结构）从自己的Cache中删除。

为了实现SI Message这一功能，PostgreSQL在共享内存中开辟了shmInvalBuffer记录系统中所发出的所有无效消息以及所有进程处理无消息的进度。shmInvalBuffer是一个全局变量，其数据类型为SISeg。

```
1.  /* Shared cache invalidation memory segment */
2.  typedef struct SISeg {
3.      /* General state information */
4.      int      minMsgNum; /* oldest message still needed */
5.      int      maxMsgNum; /* next message number to be assigned */
6.      int      nextThreshold; /* # of messages to call SICleanupQueue */
7.      int      lastBackend; /* index of last active procState entry, +1 */
8.      int      maxBackends; /* size of procState array */
9.      slock_t   msgnumLock; /* spinlock protecting maxMsgNum */
10.     /* Circular buffer holding shared-inval messages */
11.     SharedInvalidationMessage buffer[MAXNUMMESSAGES];
12.     /* Per-backend invalidation state info (has MaxBackends entries). */
13.     ProcState  procState[FLEXIBLE_ARRAY_MEMBER];
14. } SISeg;
15. static SISeg *shmInvalBuffer; /* pointer to the shared inval buffer */
```

在shmInvalBuffer中，无效消息存储在由Buffer字段指定的定长数组中（其长度MAXNUMMESSAGES预定义为4096）。该数组中每个元素存储一个无效消息，也可以称该数组为无效消息队列。无效消息队列实际是一个环状结构，最初数组为空时，新来的无效消息从前向后依次存放在数组的元素中，当数组被放满之后，新的无效消息将回到Buffer数组的头部开始插入。

minMsgNum字段记录Buffer中还未被所有进程处理的无效消息编号中的最小值，maxMsgNum字段记录下一个可以用于存放新无效消息的数组元素下标。实际上，minMsgNum指出了Buffer中还没有被所有进程处理的无效消息的下界，而maxMsgNum则指出了上界，即编号比minMsgNum小的无效消息是已经被所有进程处理完的，而编号大于等于maxMsgNum的无效消息是还没有产生的，而两者之间的无效消息则是至少还有一个进程没有对其进行处理。因此在无效消息队列构成的环中，除了minMsgNum和maxMsgNum之间的位置之外，其他位置都可以用来存放新增加的无效消息。

```
1.  typedef union
2.  {
3.      int8      id; /* type field --- must be first */
4.      SharedInvalCatcacheMsg cc;
5.      SharedInvalCatalogMsg cat;
6.      SharedInvalRelcacheMsg rc;
7.      SharedInvalSmgrMsg sm;
8.      SharedInvalRelmapMsg rm;
9.      SharedInvalSnapshotMsg sn;
10. } SharedInvalidationMessage;
```

PostgreSQL在shmInvalBuffer中用一个ProcState数组（procState字段）来存储正在读取无效消息的进程的读取进度，该数组的大小与系统允许的最大进程数MaxBackends有关，在默认情况下这个数组

的大小为100（系统的默认最大进程数为100，可在postgresql.conf中修改）。ProcState记录了PID为procPid的进程读取无效消息的状态，其中nextMsgNum的值介于shmInvalBuffer的minMsgNum值和maxMsgNum值之间。

```
1.  /* Per-backend state in shared invalidation structure */
2.  typedef struct ProcState {
3.      /* procPid is zero in an inactive ProcState array entry. */
4.      pid_t   procPid; /* PID of backend, for signaling */
5.      PGPROC  *proc;   /* PGPROC of backend */
6.      /* nextMsgNum is meaningless if procPid == 0 or resetState is true. */
7.      int     nextMsgNum; /* next message number to read */
8.      bool    resetState; /* backend needs to reset its state */
9.      bool    signaled; /* backend has been sent catchup signal */
10.     bool    hasMessages; /* backend has unread messages */
11.     /* Backend only sends invalidations, never receives them. This only makes
12.      * sense for Startup process during recovery because it doesn't maintain a
13.      * relcache, yet it fires inval messages to allow query backends to see
14.      * schema changes. */
15.     bool    sendOnly; /* backend only sends, never receives */
16.     /* Next LocalTransactionId to use for each idle backend slot. We keep
17.      * this here because it is indexed by BackendId and it is convenient to
18.      * copy the value to and from local memory when MyBackendId is set. It's
19.      * meaningless in an active ProcState entry. */
20.     LocalTransactionId nextLXID;
21. } ProcState;
```

1.3、空闲空间管理

1.3.1、VACUUM机制

Postgres使用标记删除的方式来处理元组的删除。VACUUM操作将会把这类元组所占用的空间置为可用。

当修改元组属性、执行重建索引等操作时，特别是当进行多次更新、删除操作时，磁盘上会出现很多无效的元组，占据了很多磁盘空间并且会导致系统性能下降，这时就需要进行VACUUM操作清除掉这些“垃圾”。

很明显，那些经常更新或者删除元组的表需要比那些较少更新的表清理得更频繁一些。PostgreSQL开启了一个辅助进程AutoVacuum来执行自动清理工作，以确保数据库的一致性和性能。自动Vacuum对于高事务率的系统尤其重要，因为它可以防止旧版本过多导致的磁盘空间浪费和查询性能下降。

VACUUM 操作的具体实现定义在文件Vacuum.c中。根据用户输入的命令，VACUUM分为两种:一种为Full VACUUM，它会对表进行完全清理;一种为Lazy VACUUM，它仅标记无效数据空间为可用。Lazy Vacuum适合日常维护，可以在不影响用户正常使用的情况下持续进行，保持数据库的良好运行状态。VACUUM FULL 则是一个更为激进且资源消耗较大的操作，通常仅在表有显著的空间浪费或者需要紧急释放磁盘空间时才使用，并且由于其会锁定整个表，应当在数据库维护窗口内执行，以免影响到正常服务。

```

1947 static bool
1948 vacuum_rel(Oid relid, RangeVar *relation, VacuumParams *params,
1949            BufferAccessStrategy bstrategy)
1950 {
1951     LOCKMODE lmode;
1952     Relation rel;
1953     LockRelId lockrelid;
1954     Oid priv_relid;
1955     Oid toast_relid;
1956     Oid save_userid;
1957     int save_sec_context;
1958     int save_nestlevel;
1959
1960     Assert(params != NULL);
1961
1962     /* Begin a transaction for vacuuming this relation */

```

通过在命令中指定VACUUM的类型，在vacuum_rel函数中会使用不同类型的VACUUM，然后调用相应的接口函数。

1.3.1.1、GUC 参数对VACUUM行为的影响（补充）

在 PostgreSQL 中，VACUUM 命令负责清理数据库，回收空间并更新统计信息。为了更好地控制 VACUUM 的行为，PostgreSQL 提供了一系列 GUC 参数，允许用户根据实际需求进行调整。以下是一些重要的 GUC 参数及其作用：

```

if (freeze_min_age < 0)
    freeze_min_age = vacuum_freeze_min_age;
freeze_min_age = Min(freeze_min_age, autovacuum_freeze_max_age / 2);
Assert(freeze_min_age >= 0);

/* Compute FreezeLimit, being careful to generate a normal XID */
cutoffs->FreezeLimit = nextXID - freeze_min_age;
if (!TransactionIdIsNormal(cutoffs->FreezeLimit))
    cutoffs->FreezeLimit = FirstNormalTransactionId;
/* FreezeLimit must always be <= OldestXmin */
if (TransactionIdPrecedes(cutoffs->OldestXmin, cutoffs->FreezeLimit))
    cutoffs->FreezeLimit = cutoffs->OldestXmin;

```

1. 事务 ID (XID) 年龄管理:

vacuum_freeze_min_age: 该参数控制 XID 冻结的最小年龄。当一个元组的 XID 比当前 XID 减去 vacuum_freeze_min_age 更旧时，VACUUM 将尝试将其冻结。冻结的元组不会被正常的事务可见性检查考虑，从而提高查询性能。

vacuum_freeze_table_age: 该参数控制整个表的 XID 冻结年龄。当一个表的 relfrozenxid（表示表中最旧的未冻结 XID）比当前 XID 减去 vacuum_freeze_table_age 更旧时，VACUUM 将会更加积极地冻结元组，以防止 XID 回绕问题。

```

1172     */
1173     if (multixact_freeze_min_age < 0)
1174         multixact_freeze_min_age = vacuum_multixact_freeze_min_age;
1175     multixact_freeze_min_age = Min(multixact_freeze_min_age,
1176                                   effective_multixact_freeze_max_age / 2);
1177     Assert(multixact_freeze_min_age >= 0);
1178
1179     /* Compute MultiXactCutoff, being careful to generate a valid value */
1180     cutoffs->MultiXactCutoff = nextMXID - multixact_freeze_min_age;
1181     if (cutoffs->MultiXactCutoff < FirstMultiXactId)
1182         cutoffs->MultiXactCutoff = FirstMultiXactId;
1183     /* MultiXactCutoff must always be <= OldestMxact */
1184     if (MultiXactIdPrecedes(cutoffs->OldestMxact, cutoffs->MultiXactCutoff))
1185         cutoffs->MultiXactCutoff = cutoffs->OldestMxact;
1186
1187     /*
1188      * Finally, figure out if caller needs to do an aggressive VACUUM or not.
1189      */
1190     /* Determine the table freeze age to use: as specified by the caller, or
1191      * the value of the vacuum_freeze_table_age GUC, but in any case not more
1192      * than autovacuum_freeze_max_age * 0.95, so that if you have e.g. nightly
1193      * VACUUM schedule, the nightly VACUUM gets a chance to freeze XIDs before
1194      * anti-wraparound autovacuum is launched.
1195      */
1196     if (freeze_table_age < 0)
1197         freeze_table_age = vacuum_freeze_table_age;
1198     freeze_table_age = Min(freeze_table_age, autovacuum_freeze_max_age * 0.95);
1199     Assert(freeze_table_age >= 0);
1200     aggressiveXIDCutoff = nextXID - freeze_table_age;
1201     if (!TransactionIdIsNormal(aggressiveXIDCutoff))
1202         aggressiveXIDCutoff = FirstNormalTransactionId;
1203     if (TransactionIdPrecedesOrEquals(rel->rd_rel->relfrozenxid,
1204                                     aggressiveXIDCutoff))
1205         return true;
1206

```

2. 多版本并发控制 (MultiXact) ID 管理:

`vacuum_multixact_freeze_min_age`: 类似于 `vacuum_freeze_min_age`, 该参数控制 MultiXact ID 冻结的最小年龄。MultiXact ID 用于跟踪并发事务, 冻结 MultiXact ID 可以释放相关资源并提高性能。

`vacuum_multixact_freeze_table_age`: 类似于 `vacuum_freeze_table_age`, 该参数控制整个表的 MultiXact ID 冻结年龄。当一个表的 `relminmxid` (表示表中最旧的未冻结 MultiXact ID) 比当前 MultiXact ID 减去 `vacuum_multixact_freeze_table_age` 更旧时, VACUUM 将会更加积极地冻结 MultiXact ID, 以防止 MultiXact ID 回绕问题。

3. 故障安全机制:

`vacuum_failsafe_age`: 该参数用于防止 XID 回绕导致数据丢失。当一个表的 `relfrozenxid` 比当前 XID 减去 `vacuum_failsafe_age` 更旧时, VACUUM 将会强制进行清理, 即使这可能会影响性能。

`vacuum_multixact_failsafe_age`: 类似于 `vacuum_failsafe_age`, 该参数用于防止 MultiXact ID 回绕导致数据丢失。

4. 成本控制:

`autovacuum_vacuum_cost_delay` 和 `vacuum_cost_delay`: 这两个参数控制 VACUUM 在执行 I/O 操作后延迟的时间, 以避免对系统造成过大的负载。

`autovacuum_vacuum_cost_limit` 和 `vacuum_cost_limit`: 这两个参数控制 VACUUM 的成本阈值。当 VACUUM 的累积成本超过阈值时, 将会触发延迟机制。

5. 其他参数:

`vacuum_index_cleanup`: 该参数控制 VACUUM 是否清理索引中的无效条目。

`vacuum_truncate`: 该参数控制 VACUUM 是否截断表文件以释放空间。

1.3.2 Lazy VACUUM

1.3.2、Lazy VACUUM

作用:

清除死元组: 当数据库中发生更新或删除操作时, PostgreSQL遵循MVCC (多版本并发控制) 模型, 不会立即从磁盘上物理删除行记录, 而是将其标记为可回收的死元组。Lazy Vacuum的主要作用就是清理这些不再使用的死元组, 将这些不再需要的旧版本行标记为空闲空间, 使得新插入的数据可以重用这部分空间。同时, 它还会更新表的统计信息, 帮助优化器做出更好的查询计划决策。

特点:

在线进行: Lazy Vacuum可以在表上并发执行, 即在清理死元组的同时, 其他客户端仍可以读取和修改表数据, 对表施加的锁级别较低, 通常是在页级别而非整个表。

空间回收有限: 尽管清理死元组, 但它并不会立即归还这部分磁盘空间给操作系统, 而是将空间在表内部重新组织, 使得后续插入操作能够重用这些空间, 但是表文件的实际大小可能不会明显减少。

资源消耗相对较小: 相比于Full VACUUM, Lazy Vacuum通常所需资源较少, 执行速度更快, 对系统性能的影响也相对较小。

1.3.3、Full VACUUM

作用:

物理空间重组: Full Vacuum不仅清除死元组, 还对表进行完全物理重组, 类似于重建表的过程。它会创建一个新的、紧凑的表结构, 仅包含活动数据, 并且将所有的索引重建到新的表上。

彻底释放空间: 执行完VACUUM FULL后, 由于创建了一个全新的表, 原来旧表占用的空间会被完全释放回操作系统, 从而显著缩小数据库文件大小。

特点:

锁表: VACUUM FULL在执行期间会对表及其索引获取 AccessExclusiveLock 锁, 这意味着在此期间, 任何针对该表的操作都将被阻塞, 直到VACUUM FULL完成。

资源消耗大: 由于涉及到完整的数据复制和索引重建, VACUUM FULL的执行时间通常比Lazy Vacuum长得多, 同时消耗更多系统资源, 如CPU、内存和I/O。

适用于严重碎片化或空间回收需求: 只有在表存在大量死元组, 或者需要最大程度地释放磁盘空间的情况下, 才推荐使用VACUUM FULL。考虑到其对业务连续性的潜在影响, 应尽量在系统低负载时段

执行此操作。

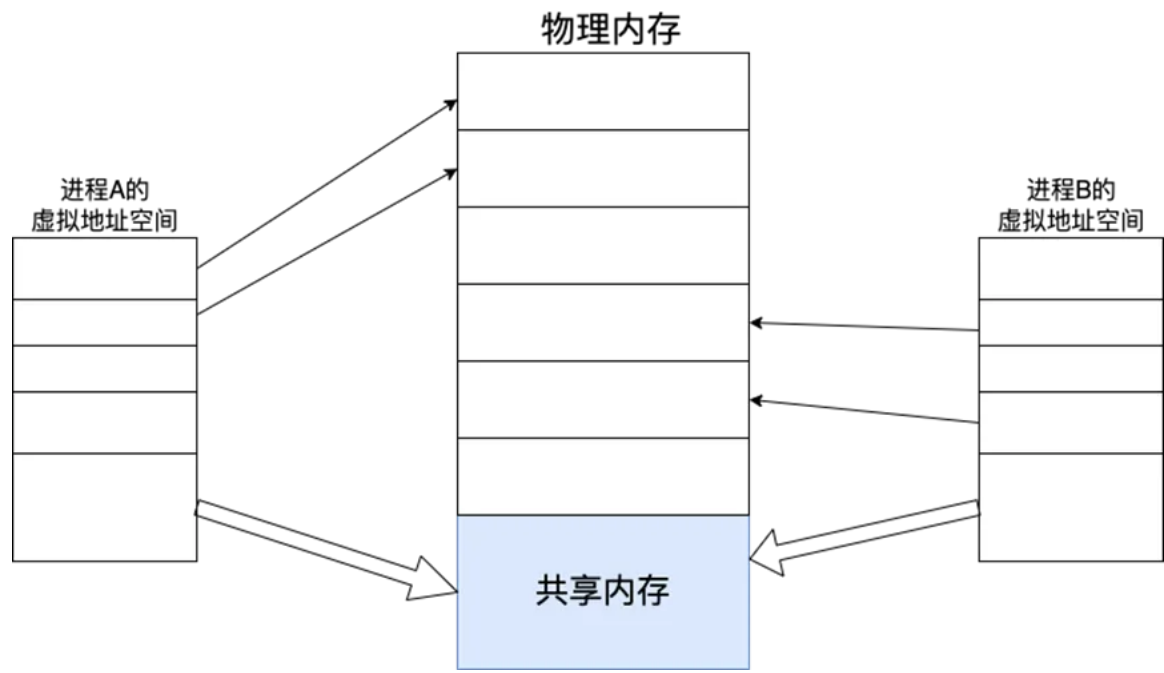
[实验：见第五实验部分\(5.1、FULL VACUUM\)](#)

1.4、进程间通信机制管理/IPC(进程间通信)

1.4.1、共享内存管理

PostgreSQL初始化过程中，系统会分配一块内存区域，它对PostgreSQL系统中的所有后端进程而言是可见的，该内存区域称为共享内存。

每个进程都有独立的虚拟空间地址，通过MMU地址转换将虚拟地址与物理地址进行映射，每个进程虚拟地址空间都会映射到不同的物理地址，每个进程在物理内存空间都是相互独立和隔离的。共享内存通过分配一块共享的物理空间，将其挂接到相互通信的进程虚拟地址空间中，实现虚拟地址到共享物理内存的映射。



1.4.2、SI Message

SI Message主要用于不同进程的Cache进行同步操作。这里主要分析一下SI Message是如何借助上面的共享内存来实现的。

shminvalBuffer。该数据结构位于共享内存中，用于记录系统中所发出的所有无效消息以及所有进程处理无效消息的进度。

关于buffer环状队列的插入：

最初数组为空时，新来的无效消息从前向后依次存放在数组的元素中，当数组被放满之后，新的无效消息将回到buffer数组的头部开始插入。

关于minMsgNum和maxMsgNum:

这两个一个可以认为是无效消息的下界，另一个可以认为是无效消息的上界。即序号比minMsgNum小的无效消息是已经被所有进程处理完的，而序号大于等于maxMsgNum的无效消息是还没有产生的，两者之间的无效消息则是至少还有一个进程没有对其进行处理。所以在buffer环中，位于minMsgNum和maxMsgNum构成的窗口之外的位置都可以用来存放新增加的无效消息。

但是，向SIMessage队列中插入无效消息时，仍然会出现空间不够的情况（此时消息队列中都是没有完全被读取完的无效消息），这时就需要清理掉一部分无效消息（当前消息数加上要插入的消息数之和超过上面的nextThreshold时就要进行清理操作）。

lowbound和minsig

因为清理操作会通知未处理完相关消息的进程，进程会抛弃在Cache中的所有元组，这会导致Cache的重载让所有进程都重载Cache的话会导致较高的I/O次数，为了减少重载Cache的次数，就有了上面最小值的计算。只要nextMsgNum值小于lowbound的进程都需要重置，即设置resetState变量为真，进程就会自动进行Cache的重载。对于nextMsgNum值在lowbound和minsig之间的进程，虽然和本次清理没有关系，但为了避免频繁的清理操作，会要求这些进程加快处理无效消息的进度。然后清理操作会找出这些进程中进度最慢的一个，向他发送PROCSIG_CATCHUP_INTERRUPT信号。进程收到SIGUSR1后会一次性处理完所有的无效消息，然后继续向下一个进度最慢的进程发送PROCSIG_CATCHUP_INTERRUPT信号

主要流程如下:

- 1.计算min,lowbound和minsig的值
- 2.对每个进程的ProcState结构进行检查，将nextMsgNum低于lowbound的进程的resetState变量设置为true，并且在nextMsgNum处于lowbound和minsig之间的进程中找出最远的(进度最慢)
- 3.重新计算threshold
- 4.向第二步找到的最远的进程发送信号

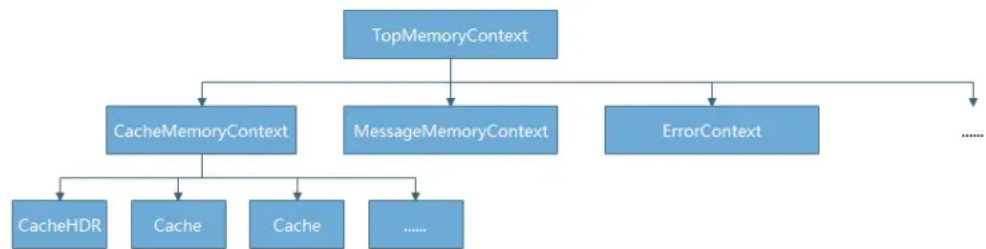
关于sendOnly:

如果为true意味着该进程只会发送无效消息，不会接收无效消息。这种情况只在恢复期间的启动过程有意义，该进程会发送无效消息从而使得其他进程能够查看架构的改变。

1.5、内存上下文 (Memory Context)

用于统一管理内存的分配和回收，从而更加有效安全地对内存空间进行管理。

系统中的内存分配操作在各种语义的内存上下文中进行，所有在内存上下文中分配的内存空间都通过内存上下文进行记录。因此可以很轻松地通过释放内存上下文来释放其中的所有内容，而不用费心地去释放其中的每一块内存，使得内存分配和释放更加快捷和可靠。



1.5.1、MemoryContext

postgresql 的每一个子进程都拥有多个私有的内存上下文。

1.5.2、内存上下文初始化与创建

内存上下文的初始化工作由函数 MemoryContextInit 来完成。

1.5.3、内存上下文中内存的分配

在Postgre SQL中， 内存的分配、 重分配和释放都是在内存上下文中进行。

1.5.4、内存上下文释放

释放内存上下文中的内存， 主要有以下三种方式：

(1)释放一个内存上下文中指定的内存片

当释放一个内存上下文中指定的内存片时， 调用函数 AllocSetFree。该函数的执行方式如下：如果指定要释放的内存片是内存块中唯一的一个内存片， 则将该内存块直接释放。 否则， 将指定的内存片加入到 Feelist 链表中以便下次分配。

(2)重置内存上下文

重置内存上下文的工作由函数 AllocSetReset 完成。在进行重置时， 内存上下文中除了在 keeper 字段中指定要保留的内存块外， 其他内存块全部释放， 包括空闲链表中的内存。keeper 中指定保留的内存块将被清空内容， 它使得内存上下文重置之后就立刻有一块内存可供使用。

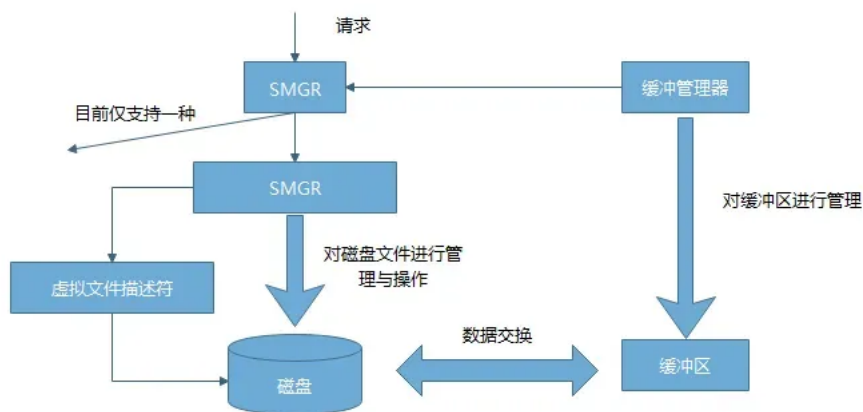
(3) 释放当前内存上下文中的全部内存块

这个工作由 AllocSetDelete 函数完成， 该函数释放当前内存上下文中的所有内存块， 包括 keeper 指定的内存块在内。但内存上下文节点并不释放， 因为内存上下文节点是在 TopMemoryContext 中申

请的内存，将在进程运行结束时统一释放。

二、外存管理

外存管理由SMGR（主要代码在smgr.c中）提供对外存操作的统一接口。SMGR负责统管各种介质管理器，会根据上层的请求选择一个具体的介质管理器进行操作。外存管理体系结构如下图所示：



2.1、表和元组组织方式

2.1.1、堆文件

元组之间不进行关联，这样的表文件称为堆文件。

a) 普通堆 (ordinary cataloged heap)

b) 临时堆 (temporary heap)

仅在会话过程中临时创建，会话结束会自动删除。

c) 序列 (SEQUENCE relation, 一种特殊的单行表)

序列则是一种元组值自动增长的特殊堆。

d) TOAST 表 (TOAST table)

在堆中要删除一个元组，理论上有两种方法：

1)直接物理删除:找到该元组所在文件块，并将其读取至缓冲区。然后在缓冲区中删除这个元组，最后再将缓冲块写回磁盘。

2)标记删除:为每个元组使用额外的数据位作为删除标记。当删除元组时，只需设置相应的删除标记，即可实现快速删除。这种方法并不立即回收删除元组占用的空间。

PostgreSQL采用的是第二种方法，每个元组的头部信息HeapTupleHeader 就包含了这个删除标记位，其中记录了删除这个元组的事务D和命令D。如果上述两个D有效，则表明该元组被删除;若无效，则表明该元组是有效的或者说没有被删除的。在第7章中将会看到这种方法对多版本并发控制也是有好处的。

2.1.2、文件存储形式

1) 由于版本更新，在postgres16.2版本下，数据页面的头信息已经达到24个字节，在bufpage.h的文件里看

```
154
155 typedef struct PageHeaderData
156 {
157     /* XXX LSN is member of *any* block, not only page-organized ones */
158     PageXLogRecPtr pd_lsn; /* LSN: next byte after last byte of xlog
159                          * record for last change to this page */
160     uint16 pd_checksum; /* checksum */
161     uint16 pd_flags; /* flag bits, see below */
162     LocationIndex pd_lower; /* offset to start of free space */
163     LocationIndex pd_upper; /* offset to end of free space */
164     LocationIndex pd_special; /* offset to start of special space */
165     uint16 pd_pagesize_version;
166     TransactionId pd_prune_xid; /* oldest prunable XID, or zero if none */
167     ItemIdData pd_linp[FLEXIBLE_ARRAY_MEMBER]; /* line pointer array */
168 } PageHeaderData;
169
170 typedef PageHeaderData *PageHeader;
171
```

其头部内容，至少包括但不限于：空闲空间的起始和结束位置；Special space 的开始位置；项指针的开始位置；页面大小和版本信息；标志信息：是否存在空闲项指针、是否所有的元组都可见、页面修改日志序列号 (LSN)、页面的校验和。用于检测页面是否损坏、页面中最老的可修剪事务ID。用于垃圾回收。

2) 除此之外，还包括了一系列linp。Linp是 ItemIdData 类型的数组，ItemIdData 类型由 lp_off、lp_flags 和 lp_len 三个属性组成。每一个 ItemIdData 结构用来指向文件块中的一个元组，其中 lp_off 是元组在文件块中的偏移量，而 lp_len 则说明了该元组的长度，lp_flags表示元组的状态（分为未使用、正常使用、HOT 重定向和已被删除四种状态）。每个 Linp 数组元素的长度为6字节。

3) 页面中未被使用的空间，用于存储新的数据。可用空间分为两部分：pd_lower 和 pd_upper 之间的空间(Freespace 空闲空间)、未使用的行指针。其中 Linp 元素从 Freespace 的开头开始分配，而新元组数据则从尾部开始分配。

4) Special space 是特殊空间，用于存放与索引方法相关的特定数据。Special space 在普通表文件块中并没有使用，其内容被置为空。只有设置索引才被使用。

2.2、表文件管理

在PostgreSQL中，每个表都用一个文件（表文件）存储，表文件以表的OID命名。

2.2.1、分页存储管理

每个表文件由多个 BLCKSZ（一个可配置的常量）字节大小的文件块组成，每个文件块又可以包含多个元组。

表文件以文件块为单位进行IO交换。一个文件块的大小为8k

2.2.2、行式存储

2.2.3、系统表

2.3、磁盘管理器

2.3.1、SMGR实现

磁盘管理器是SMGR的一种具体实现，它对外提供了管理磁盘介质的接口，其主要实现在文件 md.c中。

```
typedef struct _MdfdVec
{
    File      mdfd_vfd;          /* fd number in fd.c's pool */
    BlockNumber mdfd_segno;      /* segment number, from 0 */
} MdfdVec;

static MemoryContext MdCxt;      /* context for all MdfdVec objects */
```

mdfd_vfd:该文件所对应的 VFD（见 3.2.3 节VFD机制）。mdfd_segno :由于较大的表文件会被切成多个文件（可以称为段），因此用这个字段表示当前这个文件是表文件的第几段。mdfd_chain:指向同一表文件下一段的MdfdVec, 通过这个字段可以把表文件的各段链接起来，形成链表。

通过mdfd_chain链表使得大于操作系统文件大小（通常为2GB)限制的表文件更易被支持，因为我们可以将表文件切割成段，使每段小于2GB。

2.3.2、VFD机制

VFD机制通过合理使用有限个实际文件描述符来满足无限的VFD访问需求。

VFD机制的原理类似连接池，当进程申请打开一个文件时，总是能返回一个虚拟文件描述符，对外封装了打开物理文件的实际操作。

▼ VFD

C++

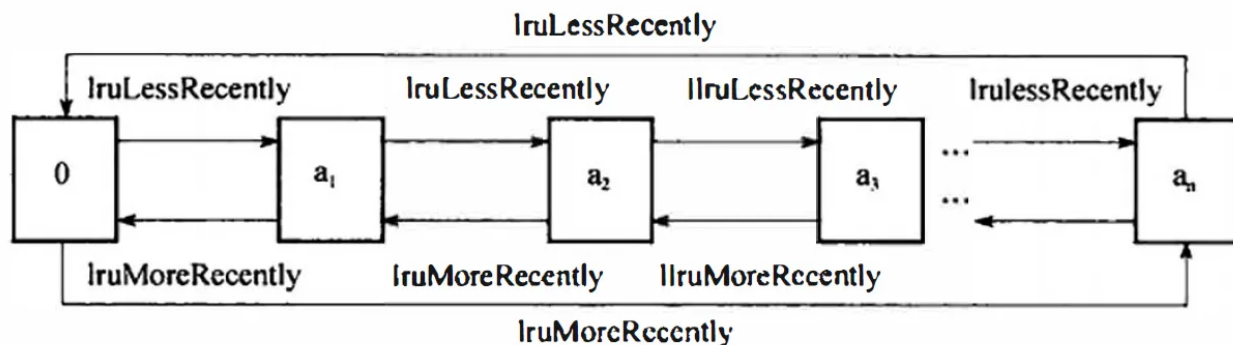
```
1  typedef struct vfd
2  {
3      int fd; /* current FD, or VFD_CLOSED if none */
4      unsigned short fdstate; /* bitflags for VFD's state */
5      ResourceOwner resowner; /* owner, for automatic cleanup */
6      File nextFree; /* link to next free VFD, if in freelist
7  */
8      File lruMoreRecently; /* doubly linked recency-of-use list
9  */
10     File lruLessRecently;
11     off_t seekPos; /* current logical file position, or -1 */
12     off_t fileSize; /* current size of file (0 if not temporary) */
13 }
14     char *fileName; /* name of file, or NULL for unused VFD */
15 /* NB: fileName is malloc'd, and must be free'd when closing the VFD
16 */
17     int fileFlags; /* open(2) flags for (re)opening the file
18 */
19     int fileMode; /* mode to pass to open(2) */
20 } Vfd;
```

VFD 机制的实现主要依赖于一个 VFD 数组，其中每个元素对应一个 VFD，而每个 VFD 包含了与打开的文件相关的信息，例如文件描述符、文件状态、文件指针位置等。通过这种方式，PostgreSQL 可以轻松地管理已经打开的文件，并且可以根据需要动态调整 VFD 数组的大小。

在 PostgreSQL 中，每个后台进程都会使用一个 LRU（Least Recently Used，最近最少使用）池来管理所有已经打开的 VFD。LRU 池通过维护一个链表结构，按照 VFD 最近的使用情况进行排序，使得最近使用的 VFD 总是排在链表的前面，而最久未使用的 VFD 则排在链表的末尾。

2.3.3、LRU池

在每一个 PostgreSQL 后台进程中都使用一个 LRU（Last Recently Used，最近最少使用）池来管理所有已经打开的 VFD，池中每个 VFD 都对应一个物理上已经打开的文件。



从LRU池里VFD的操作主要包括以下三种：

(1)从LRU池删除 VFD

该操作发生在进程使用完一个文件并关闭它时，通过LruDelete函数实现。该操作将指定的VFD从LRU池中删除，并将该VFD对应的文件关闭掉。例如，我们要对Vfd $[a_2]$ 执行LruDelete 操作，则首先将Vfd $[a_2]$ 的lruLessRecently指向Vfd $[a_3]$ ，将Vfd $[a_3]$ 的lruMoreRecently指向Vfd $[a_1]$ 。这样就将Vfd $[a_2]$ 从LRU池中删除了，如果Vfd $[a_2]$ 的fdstate被置为FD_DIRTY，则要先将Vfd $[a_2]$ 对应的文件同步回磁盘，再清掉 FD_DIRTY位。接着将Vfd $[a_2]$ 的seekpos置为该文件当前读写的指针位置，最后将Vfd $[a_2]$ 对应的文件关闭掉并将Vfd $[a_2]$ 中的fd置为VFD_CLOSED。这样Vfd $[a_2]$ 就变为空闲，还需要将其加入到空闲链表中。

(2)将VFD插入LRU池

该操作发生在打开一个新的VFD时，通过LruInsert函数实现。该操作将指定 VFD对应的物理文件打开，并将该VFD插入到VFD[0]之后的位置。例如，我们要插入的VFD为b，首先要根据b中的fd字段判断对应的物理文件 是否已经打开了，如果没有，则根据b中的fileName打开此文件，并插入到LRU池中。插入时要将b的lruMore-

Recently指向Vfd[0]。Vfd[0]的lruLessRecently指向b，然后 b的lruLessRecently指向Vfd $[a_1]$ ，Vfd $[a_1]$ 的lruMoreRecently指向b。

(3)删除 LRU池尾的VFD

该操作通过ReleaseLruFile函数实现，它将LRU池中末尾的那个VFD删除。当LRU池已满而此时又要打开 新的文件时，就需要执行ReleaseLruFile 操作，将池中末尾的VFD（最少使用的VFD）删掉，这样新打开的VFD 就可以插入到LRU中。注意，这里被删除的VFD仅仅只是从LRU 池中脱链并关闭其对应的物理文件，VFD结构本身并不做其他修改和删除。因为进程后面的操作还可能会用到该VFD所对应的物理文件。当再次需要访问一个LRU池之外的VFD时，需要先根据VFD中记录的文件打开标志打开其对应的物理文件，然后根据VFD中记录的读写指针位置将物理文件描述符的读写指针移动到正确的位置，最后还要把该VFD重新插入到LRU池中。

2.4、空间管理

2.4.4、空闲表空间映射

对于每个表文件（包括系统表在内），同时创建一个名为“关系表OID_fsm”的文件，用于记录该表的空闲空间大小，称之为空闲空间映射表文件(FSM)

FSM 文件的主要作用是跟踪和管理表空间内的空闲区域。这有助于数据库系统更高效地分配和回收空间，尤其是在频繁进行插入、更新和删除操作的场景下。通过使用 FSM，PostgreSQL 可以快速找到足够大的连续空间来满足新数据的需求，而无需进行耗时的磁盘碎片整理。

FSM特点：

- 1) 每个堆和索引关系（除了哈希索引）都有一个FSM用来跟踪关系中的可用空间
- 2) FSM作为一个独立的分支存放在主要关系数据旁边，文件名格式为 `filenode number 加 _fsm` 后缀，如下图oid为16384的数据库里的oid为12203的表，它的FSM文件名为12203_fsm。

```
lzh@iZwz98qdx9tvkkfsuhp66eZ:~/postgresql/pgdata/base/16384$ ls
112      1255_vm  2608_fsm  2661      2830_vm  3394      3603
113      1259    2608_vm  2662      2831     3394_fsm  3603_fsm
12203    1259_fsm 2609     2663      2832     3394_vm  3603_vm
12203_fsm 1259_vm  2609_fsm  2664      2832_vm  3395     3604
12203_vm 1417     2609_vm  2665      2833     3439     3605
12205    1417_vm  2610     2666      2834     3439_vm  3606
12207    1418     2610_fsm  2667      2834_vm  3440     3607
```

- 3) FSM被组织成一棵FSM页的树，底层的FSM页存储了每一个 heap (or index) page 中可用的空闲空间，每个页对应一个字节。上层FSM页面则聚集来自于下层页面的信息
- 4) 每个FSM页里是一个数组表示的二叉树，每个节点一个字节。每个叶节点表示一个堆页面或者一个下层FSM页面。在每一个非叶节点中存储了它孩子节点中的最大值

FSM中存储的并不是实际的空闲空间大小，而是用一个字节来表示一个范围内的空闲空间大小，如下表所示：

字节	空闲空间范围 (字节)
0	0 ~ 31
1	32 ~ 63
...	...
255	8164 ~ 8192

为了更快的查找，FSM文件也不是使用数组存储每个页的空闲空间，而是使用了一个三层树结构。第0层和第1层是辅助层，第2层用于实际存放各表页中的空闲空间字节位。

FSM页内，有两个基本操作：查找和更新

查找：

1) 要在一个FSM页里找一个有 X 空闲空间的页，也就是 $n \geq X$ 的叶子节点，只需要从根节点开始，选一个大于 $n \geq X$ 的孩子作为下一步，一直遍历到叶子节点即可。

2) 但从 `fsm_set_avail()` 看不完全是这样的。在一个FSM页里找一个有 X 空闲空间的页，它会先看根节点是否 $\geq X$ ，如果有再从 `fp_next_slot` 开始找：每一步向右移再爬上父节点，在有足够free space的节点停下，停下后在用开始说得方法向下走。（`fp_next_slot`有两个作用，在找底层FSM页时，每次找到后会指向找到的`slot+1`，以分散FSM搜索返回的页面。在找上层FSM页时，找到后指向找到的`slot`，那下次也从这里开始找，可以利用OS的预取和批量写入的优化）

更新：

1) 要更新一个页的空闲空间为 X，首先更新对应的叶子节点，然后不断向上走，维护父节点为两个孩子的最大值，直到一个父节点的值不变即可停下

2.4.5、可见性映射表

PostgreSQL中为了实现多版本并发控制，当事务删除或更新元组时，并非从物理上删除，而是通过将其标记为无效的方式进行标记删除，最终对这些无效元组的清理操作需要调用VACUUM来完成。

对于每个表文件，其对应的VM文件命名为：“关系表OID_vm”。对该文件的操作在`visibility-map.c`文件中进行了定义。

```
12
13 * INTERFACE ROUTINES
14 *     visibilitymap_clear - clear bits for one page in the visibility map
15 *     visibilitymap_pin   - pin a map page for setting a bit
16 *     visibilitymap_pin_ok - check whether correct map page is already pinned
17 *     visibilitymap_set   - set a bit in a previously pinned page
18 *     visibilitymap_get_status - get status of bits
19 *     visibilitymap_count  - count number of bits set in visibility map
20 *     visibilitymap_prepare_truncate -
21 *         prepare for truncation of the visibility map
22 *
```

以下是几个与可见性映射表操作相关的函数：

`visibilitymap_clear`：清除指定页在VM中的特定状态标记。

`visibilitymap_pin`：锁定映射页以便设置比特位。

`visibilitymap_pin_ok`：验证指定映射页是否已被正确锁定。

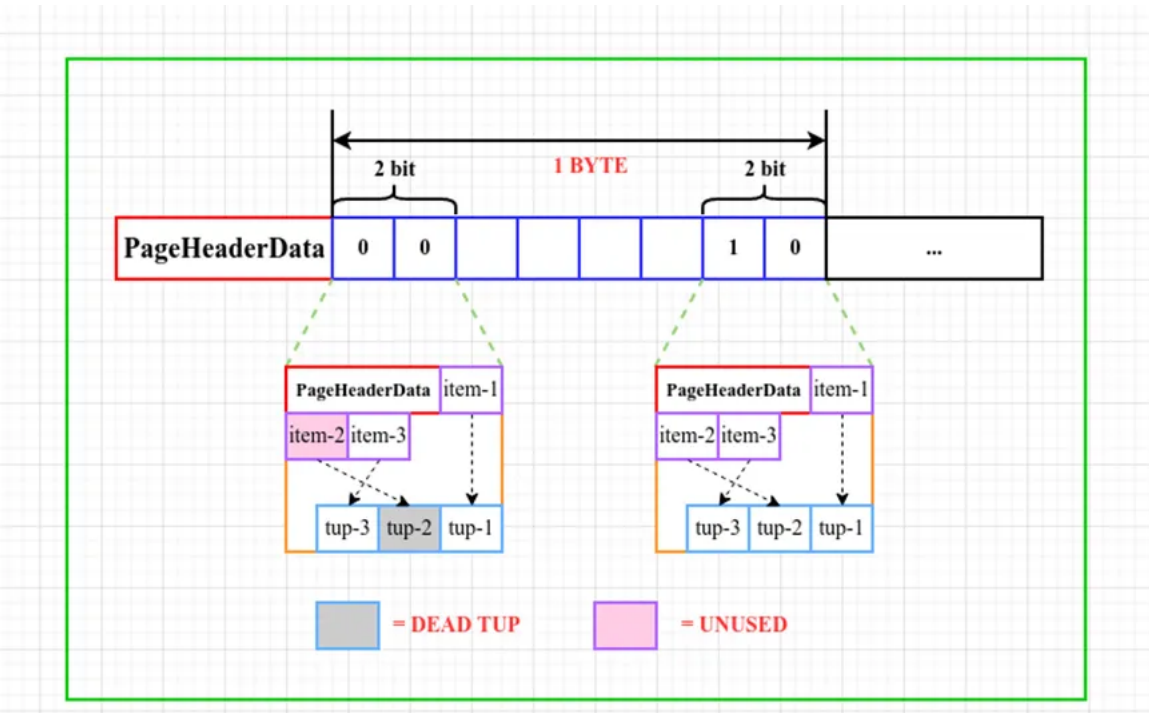
`visibilitymap_set`：在已锁定的映射页中设置比特位。

`visibilitymap_get_status`：查询映射中指定页面的比特位状态。

`visibilitymap_count`：统计VM中被设置的比特位总数。

`visibilitymap_prepare_truncate`：为截断VM做好准备工作，包括释放资源或调整大小。

每当事务对表块中的元组进行更新或删除时，对应的VM文件中标志位将被清零，设置标志位前需锁定VM页面，以避免并发修改导致VACUUM判断失误。



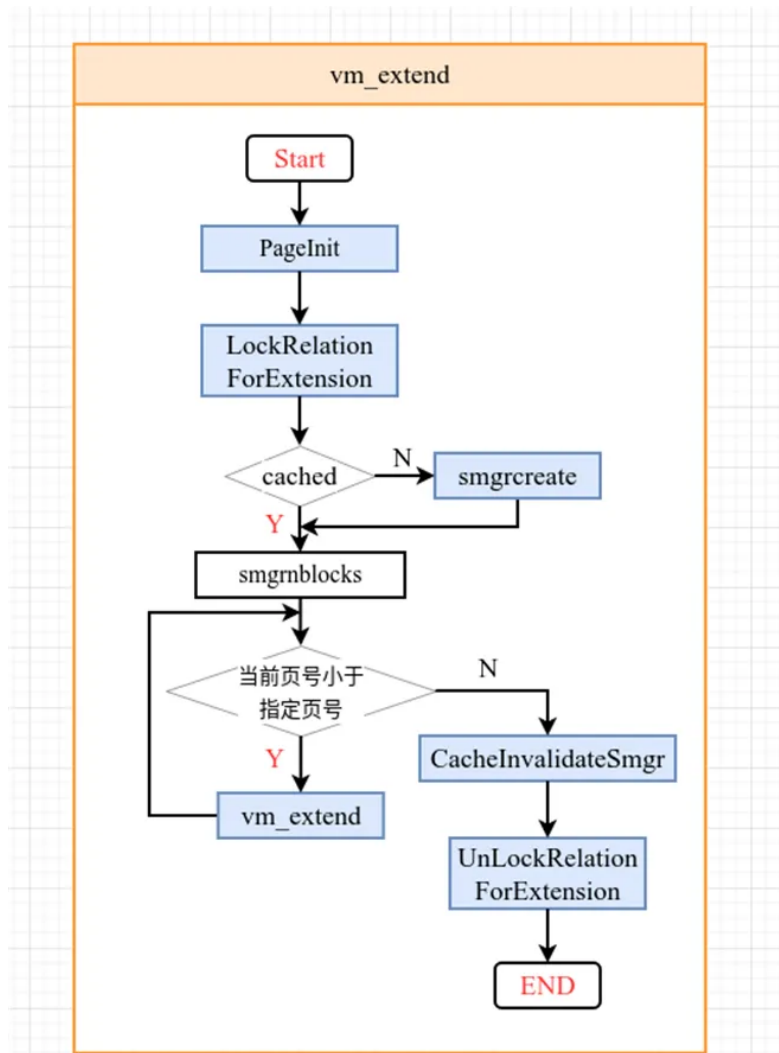
当标志位为1时，VACUUM会忽略扫描对应的表块，所以能大大提高VACUUM的效率。由于VM文件不跟踪索引，所以对索引的操作还是需要完全扫描。

当对某个表块中的元组进行更新或删除后，该表块在VM文件中对应的标志位将被置为0。在设置标志位时，需要对其对应的VM页面加锁。

可以通过create extension pg_visibility;select pg_visibility_map(pg_static); 语句查询状态可见性。pg_visibility_map () 函数的实现原理如下：

- 1) 首先打开 relation即VM文件，随后执行check_relation_relkind函数，此处只支持RELKIND_RELATION、RELKIND_INDEX、RELKIND_MATVIEW、RELKIND_SEQUENCE、RELKIND_TOASTVALUE几种类型。
- 2) 再通过pg_visibility_tupdesc组装出tup的描述。一般情况下，tupdesc能够直接在系统表里获取。而此处由于数据格式固定，因此需要自行生成。
- 3) 然后通过visibilitymap_get_status获取页面的可见性信息。函数逻辑为：传入blocknumber并计算其偏移量，然后读取对应的buffer和内容并返回状态信息。
- 4) 将返回的状态信息与VISIBILITYMAP_ALL_VISIBLE、VISIBILITYMAP_ALL_FROZEN分别进行对比得到两个布尔值，将布尔值组装成DATUM值返回，并解析成(1,t,t)、(2,f,f)形式以显示可见状态。

当访问的vm页在文件中不存在时，此时需调用vm_extend函数扩展新页并完成相应的初始化工作，其执行流程图如下：



- 1) 首先页面初始化，填充PageHeader结构体pd_lower、pd_upper/和flag初始信息；
- 2) 获取relation的extension锁，防止其他进程进行同样的扩展工作；
- 3) 如果文件不存在，则调用 smgrcreate进行创建，反之进入第4) 步；
- 4) 获取当前vm块号，如果当前块号小于指定快号，则需在此调用vm_extend进行扩展（递归调用）；
- 5) 向其他进程发送无效消息强制其关闭对rel的引用，其目的是避免其他进程对此文件的create或者extension,因为这写操作容易发生。
- 6) 最后释放锁资源；

接口函数

该函数的主要功能是设置可见性标识位，其执行流程如下：

- 1) 首先进行安全性校验，判断传入的heap buf 和 vmbuf是否有效以及buf中缓存页是否一一对应；

2) 获取 VM 页内容首地址 (跳过 PageHeaderData) , 获取 vmbuf 的 BUFFER_LOCK_EXCLUSIVE;

3) 如果之前没有设置过相应的标识位, 进行如下操作:

(1) 进入临界区, 在指定bit位设置信息, 将vmbuf标记为脏;

(2) 写WAL日志, 如果开启wal_log_hints, 需要将此日志号的LSN更新至heap 页后中; 最后更新vmbuf缓存页的LSN, 并退出临界。

4) 释放vmbuf 持有的排他锁。

2.5、大的数据存储管理

大数据的存储使用TOAST 机制和大对象机制来实现, 前者主要用千变长字符串, 后者则主要用于大尺寸的文件。

要理解TOAST, 我们要先理解页 (BLOCK) 的概念。在PG 中, 页是数据在文件存储中的基本单位, 其大小是固定的且只能在编译期指定, 之后无法修改, 默认的大小为8KB。同时, PG 不允许一行数据跨

页存储。那么对于超长的行数据, PG 就会启动TOAST, 将大的字段压缩或切片成多个物理行存到另一张系统表中 (TOAST 表), 这种存储方式叫行外存储。

2.5.1、TOAST表结构



● 前4 字节 (32bit) 称为长度字, 长度字后面存储具体的内容或一个指针。

● 长度字的高2bit 位是标志位, 后面的30bit 是长度值 (表示值的总长度, 包括长度字本身, 以字节计) 。

● 由长度值可知TOAST 数据类型的逻辑长度最多是30bit, 即1GB($2^{30}-1$ 字节) 之内。

- 前2bit 的标志位，一个表示压缩标志位，一个表示是否行外存储，如果两个都是零，那么表示既未压缩也未行外存储。

TOAST 表有三个字段：

- chunk_id —— 用来表示特定TOAST 值的OID，可以理解为具有同样chunk_id 值的所有行组成原表（这里的blog）的TOAST字段的一行数据。

- chunk_seq —— 用来表示该行数据在整个数据中的位置。

- chunk_data —— 该Chunk 实际的数据。

2.5.2、TOAST操作

在PG 中每个表字段有四种TOAST 的策略：

- PLAIN —— 避免压缩和行外存储。只有那些不需要TOAST 策略就能存放的数据类型允许选择（例如int 类型），而对于text 这类要求存储长度超过页大小的类型，是不允许采用此策略的。

- EXTENDED —— 允许压缩和行外存储。一般会先压缩，如果还是太大，就会行外存储。这是大多数可以TOAST 的数据类型的默认策略。

- EXTERNAL —— 允许行外存储，但不许压缩。这让在text 类型和bytea 类型字段上的子串操作更快。类似字符串这种会对数据的一部分进行操作的字段，采用此策略可能获得更高的性能，因为不需要读

取出整行数据再解压。

- MAIN —— 允许压缩，但不许行外存储。不过实际上，为了保证过大数据的存储，行外存储在其它方式（例如压缩）都无法满足需求的情况下，作为最后手段还是会被启动。因此理解为：尽量不使用行外存储更贴切。

实验：见第五实验部分(5.3、TOAST)

2.5.3、大对象

随着信息技术的发展，数据库需要存储的数据对象变得越来越大，为了解决大对象的存储 PostgreSQL提供了大对象存储机制。支持三种数据类型的存储：

1. 二进制大对象(BLOB):主要用来保存非传统数据，如图片、视频、混合媒体等。
2. 字符大对象(CLOB):存储大的单字节字符集数据，如文档等。
3. 双字节字符大对象(DBCLOB):用于存储大的双字节字符集数据，如变长双字节字符图形字符串。

这些类型可以保存诸如音视频、图片以及文本等大数据量的对象，最大可支持2G的尺寸。

所有的大对象都存在一个名为pg_largeobject的系统表中。每一个大对象还在系统表

pg_largeobject_metadata中有一个对应的项。大对象可以通过类似于标准文件操作的读/写API来进行创建、修改和删除。

```
SQL |
1 postgres=# create table image(name text, raster oid);
2 CREATE TABLE
3 postgres=# select lo_creat(-1);
4 lo_creat
5 -----
6 49727
7 (1 row)
```

创建一个新的大对象。其返回值是分配给这个新大对象的OID或者InvalidOid (0) 表示失败。

```
SQL |
1 postgres=# select lo_create(43213);
2 lo_create
3 -----
4 43213
5 (1 row)
```

尝试创建OID为43213的大对象，返回值是分配给新大对象的OID或InvalidOid (0) 表示发生错误。lo_create是从PostgreSQL 8.1版本中开始提供的函数，如果该函数在旧服务器版本上运行，它将失败并返回InvalidOid。

```
SQL |
1 postgres=# select lo_unlink(43213);
2 lo_unlink
3 -----
4 1
5 (1 row)
```

TOAST 和大对象虽然都是对大数据进行存储的技术，但两者有一定的区别。

首先，TOAST 用于存储变长的数据，如 VARCHAR 变量等。只要表中有变长的数据类型，都会自动地创建 TOAST 表，但只有存入的数据长度超过 2KB，才会触发 TOAST 存储机制。而大对象操作是客户端通过代码或者 SQL 命令调用的。因此，两者的第一个区别在于 TOAST 是可变长数据类型的一种大数据存储机制，属于自动触发机制；而大对象属于用户手动调用机制。

其次，TOAST 中的数据不能丢失，一旦丢失则会报错；而大对象中允许数据丢失，如果丢失，则用 0 替代。

第三，文件不适合使用 TOAST 技术进行存储，需要将文件以二进制方式读出，然后将二进制当作字符串存储到变长数据类型的属性中。而读取时则需要将数据读取出来，然后再转变成文件。而大对象操作不一样，它直接将文件作为一个对象存储到大对象表中，读取的时候也直接读取成一个文件，大对象对文件的存储非常容易通过编程来实现。

最后，TOAST 和大对象操作保存大数据时都是采用了将数据切成片段存储到表中的方式。但 TOAST 提供了线外和压缩两种存储机制，而大对象只是对数据不做处理直接存储。

三、表操作和元组操作

3.1、表操作

表的操作由 heapam.c 提供接口。有两种操作表的方式——以表名进行操作/以表的 OID 进行操作。但实际上，以表名进行操作的函数，是先通过表的名称获取 OID，再调用 OID 进行擦欧总的函数。

3.1.1、打开表

并不是打开具体的物理文件，仅仅返回表的 RelationData 结构体源码位于 table.c

```
29
30  /* -----
31   *      table_open - open a table relation by relation OID
32   *
33   *      This is essentially relation_open plus check that the relation
34   *      is not an index nor a composite type. (The caller should also
35   *      check that it's not a view or foreign table before assuming it has
36   *      storage.)
37   * -----
38   */
39  Relation
40  table_open(Oid relationId, LOCKMODE lockmode)
41  {
42      Relation    r;
43
44      r = relation_open(relationId, lockmode);
45
46      validate_relation_kind(r);
47
48      return r;
49  }
```


如下函数(validate_relation_kind)根据表的OID打开，调用relation_open函数，该函数根据表的OID从RelCache中找到相应的关系描述符，并将引用计数加一。如果是第一次打开，会在RelCache中创建一个新的RelationData，并将引用计数设置为1。然后根据第二个参数，即锁的类型进行上锁。

```
/* -----  
 * validate_relation_kind - check the relation's kind  
 *  
 * Make sure relkind is not index or composite type  
 * -----  
 */  
  
static inline void  
validate_relation_kind(Relation r)  
{  
    if (r->rd_rel->relkind == RELKIND_INDEX ||  
        r->rd_rel->relkind == RELKIND_PARTITIONED_INDEX ||  
        r->rd_rel->relkind == RELKIND_COMPOSITE_TYPE)  
        ereport(ERROR,  
                (errcode(ERRCODE_WRONG_OBJECT_TYPE),  
                 errmsg("cannot open relation \"%s\"",  
                        RelationGetRelationName(r)),  
                 errdetail_relkind_not_supported(r->rd_rel->relkind)));  
}
```

具体代码解释可参考博客

https://blog.csdn.net/weixin_45644897/article/details/121746974

3.1.2、扫描表

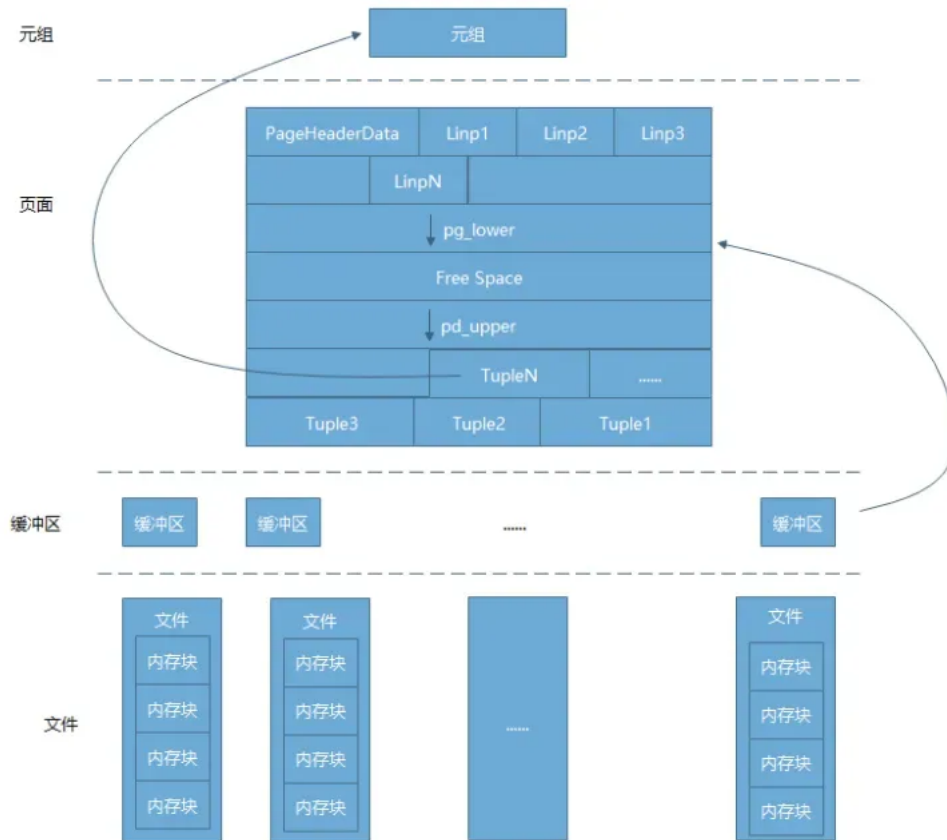
在任何关系数据库引擎中，都需要生成一个尽可能好的计划，该计划耗费最少的时间和最小的资源进行查询。通常，数据库都以树结构生成计划，每个计划树的叶节点称为表扫描节点，这个节点用于从基表中获取数据的使用的算法。扫描表的目的是从表中**获得需要的元组**。

当前postgres支持以下方式从表中获取数据：Sequential Scan、Index Scan、Index Only Scan、Bitmap Scan、TID Scan

每一种扫描方式都是有用的，取决于查询和参数配置，比如：表的cardinality、表的选择性、磁盘io代价、随机io代价、顺序io代价等等。

[实验：见第五实验部分\(5.2、扫描表\)](#)

顺序扫描如下图

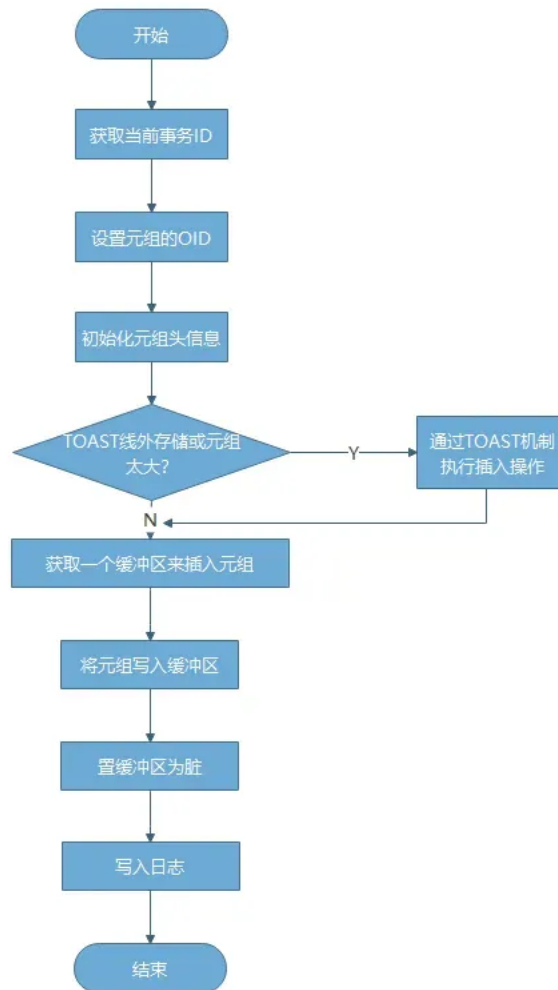


3.1.3、同步扫描

一种新的顺序扫描策略，解决以下类似情况：同时有三个扫描A,B,C对T表进行顺序扫描。理论上来说，A扫描过的文件块都应该缓存再缓冲池中，这样比A进度慢的B和C应该都能够从缓冲池中找到自己需要的文件块，从而避免对于物理文件的直接访问。但是再实际情况中会有大量的并发进程同时访问数据库，缓冲池的大小有限不能存放所有的访问数据，因此旧的数据会被新的替换。所以A的访问数据很有可能在B需要访问的时候已经被替换，那么B就需要读取物理文件。

3.2、元组操作

对元组的操作包括插入、删除和更新三种基本操作，这三种操作都是把元组当作一个整体进行处理。除些之外，在 `heaptuple.cpp` (OG 中) 这个文件中还实现了元组内部结构的相关操作，包括元组的构造修改、分解、复制、释放等作。



一个完整的元组信息将对应一个 `HeapTupleData` 结构和一个 `TupleDesc` 结构，在 `HeapTupleData` 中还包含一个 `HeapTupleHeaderData` 结构。`TupleDesc` 是关系结构 `RelationData` 的一部分，也称为元组描述符，它记录了与该元组相关的全部属性模式信息。通过元组描述符可以读取磁盘中的无格式数据，并根据元组描述符构造出元组的各个属性值，元组描述符的结构如下所示（路径：`src/include/access/tupdesc.h`）。

```

typedef struct TupleDescData
{
    int          natts;          /* number of attributes in the tuple */
    Oid          tdtypeid;       /* composite type ID for tuple type */
    int32        tdtypmod;       /* typmod for tuple type */
    int          tdrefcount;     /* reference count, or -1 if not counting */
    TupleConstr *constr;        /* constraints, or NULL if none */
    /* attrs[N] is the description of Attribute Number N+1 */
    FormData_pg_attribute attrs[FLEXIBLE_ARRAY_MEMBER];
    TupleDescData;
}
typedef struct TupleDescData *TupleDesc;
  
```

```
typedef struct HeapTupleData
{
    uint32          t_len;                /* length of *t_data */
    ItemPointerData t_self;               /* SelfItemPointer */
    Oid              t_tableOid;          /* table the tuple came from */
#define FIELDNO_HEAPTUPLEDATA_DATA 3
    HeapTupleHeader t_data;               /* -> tuple header and data */
} HeapTupleData;

typedef HeapTupleData *HeapTuple;
```

HeapTupleData 结构体中包含一个 HeapTupleHeaderData 类型的字段 t_data，该字段指向元组的头部信息。这种组合允许数据库系统在需要时访问元组的头部信息和数据，以执行操作，如插入、更新、删除或检索元组。

```
struct HeapTupleHeaderData
{
    union
    {
        HeapTupleFields t_heap;
        DatumTupleFields t_datum;
        t_choice;
    }

    ItemPointerData t_ctid;               /* current TID of this or newer tuple (or a
                                           * speculative insertion token) */

    /* Fields below here must match MinimalTupleData! */

#define FIELDNO_HEAPTUPLEHEADERDATA_INFOMASK2 2
    uint16          t_infomask2;          /* number of attributes + various flags */

#define FIELDNO_HEAPTUPLEHEADERDATA_INFOMASK 3
    uint16          t_infomask;           /* various flag bits, see below */

#define FIELDNO_HEAPTUPLEHEADERDATA_HOFF 4
    uint8           t_hoff;               /* sizeof header incl. bitmap, padding */

    /* ^ - 23 bytes - ^ */

#define FIELDNO_HEAPTUPLEHEADERDATA_BITS 5
    bits8           t_bits[FLEXIBLE_ARRAY_MEMBER]; /* bitmap of NULLs */
```

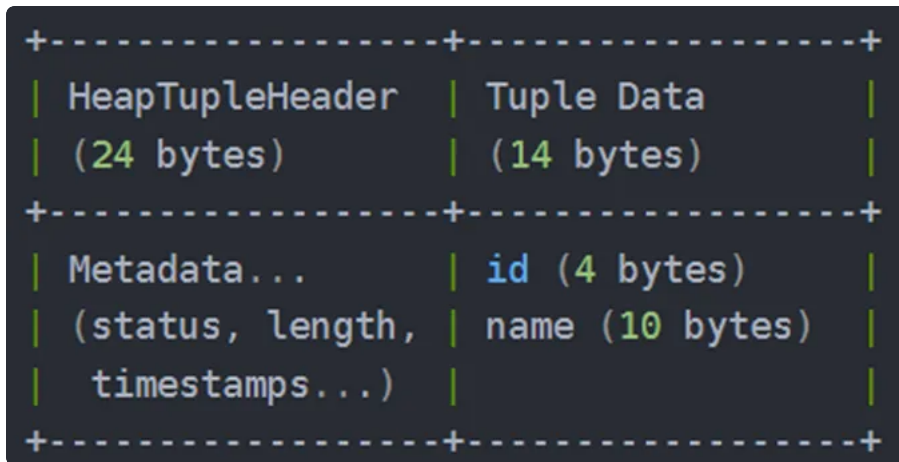
在数据结构中并没有出现存储元组实际数据的属性，这是因为 PostgreSQL（OG 可以理解为 PG 的一个分支）通过编程技巧，巧妙地将元组的实际数据存放在 HeapTupleHeaderData 结构后面的空间中。

1.数据结构的布局：在 PostgreSQL 中，HeapTupleHeaderData 结构用于存储有关元组的元数据信息，比如它的长度、状态信息、时间戳等。这个结构体本身并不包含元组的实际数据（即字段值）。

2.没有显式指针：在 HeapTupleHeaderData 结构中，通常不会有一个显式的指针来指向元组的数据部分。相反，数据紧随元组头部结构存储。这意味着一旦你有了指向 HeapTupleHeaderData 的指针，你可以通过计算偏移量来访问实际的数据。

3.优势：这种方法的优势在于效率和简洁性。它减少了额外的指针解引用和内存分配，从而提高了数据访问的效率。同时，这种布局也使得数据的存储更加紧凑，减少了内存占用。

内存布局看起来像这样：



3.1.1、插入元组

3.1.1.1、heap_form_tuple 函数

在插入元组之前，我们首先要根据元组内数据和描述符等信息初始化 HeapTuple 结构，函数 heap_form_tuple 实现了这一功能。

heap_form_tuple 函数的主要目的是创建一个新的堆元组（HeapTuple）。它接受一个元组描述符（TupleDesc），这个描述符定义了元组的结构（即它有多少个属性，每个属性的类型是什么等），以及两个数组：values（存储每个属性的值）和 isnull（标记对应的值是否为 NULL）。

函数首先检查属性的数量是否超过了所允许的最大属性数量。接着，它遍历所有属性，检查是否有 NULL 值，并处理可能需要展开的压缩（toasted）属性。计算所需的总空间，并根据是否有 NULL 值或 OID 来调整空间大小。接下来，它会分配足够的内存来存放新的元组，并设置元组头部信息，包括类型、长度、属性数量等。最后，它调用 heap_fill_tuple 函数来填充元组的实际数据，并返回这个新构造的元组。

3.1.1.2、heap_fill_tuple 函数

heap_fill_tuple 函数负责将数据从 values 和 isnull 数组中加载到元组的数据部分，并设置 null 位图（如果有的话）以及反映元组数据内容的 infomask 位。

heap_fill_tuple 函数用于填充 HeapTuple 数据结构。这个过程包括以下几个关键步骤：

- 1.处理 Null 值：函数通过一个位图（如果提供了 bit 参数）来标记哪些属性是 NULL。这是通过移动位掩码和更新位图来实现的。

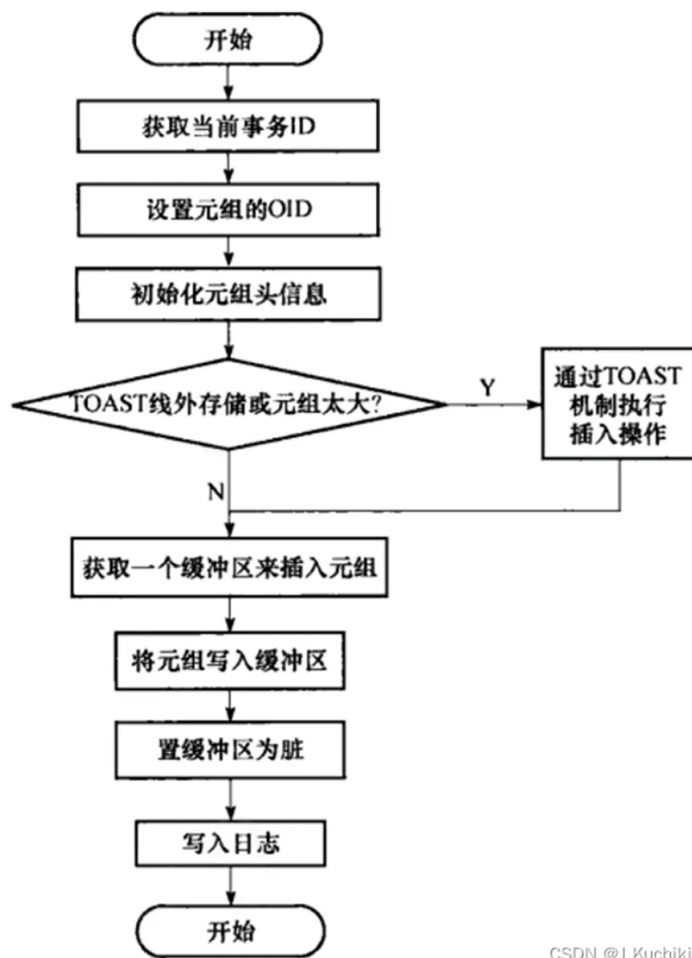
- 2.设置 Infomask：根据元组数据的特性（如是否有 NULL 值，是否有可变宽度属性等），设置 infomask 位。

3.数据复制：对于每个非 NULL 属性，函数根据其类型（按值传递、可变长度、固定长度等）将数据从 values 数组复制到元组的数据区域。这涉及到适当的内存对齐和安全的内存复制。

4.长度和类型处理：根据属性的类型（如普通数据、可变长度数据、C 字符串等），计算每个属性的数据长度，并将其复制到正确的位置。

3.1.1.3、heap_insert 函数

当完成了元组数据在内存中的构成后，下一步就可以准备向表中插入元组了。插入元组的函数接口为 heap_insert，其流程如下图所示。



CSDN @J.Kuchiki

heap_insert 函数的主要作用是将一个元组插入到数据库的一个表（即关系）中。此函数执行以下关键步骤：

- 1.准备要插入的元组，包括为元组分配 OID（对象标识符），必要时对元组进行压缩。
- 2.检查与其他事务的潜在冲突，特别是在可串行化事务中。
- 3.在适当的位置（即缓冲区）找到存储元组的空间。
- 4.在开始修改缓冲区前，进入关键操作区段，确保此过程不会因为错误而中断。
- 5.将元组实际插入到表中，并更新相关的可见性信息。

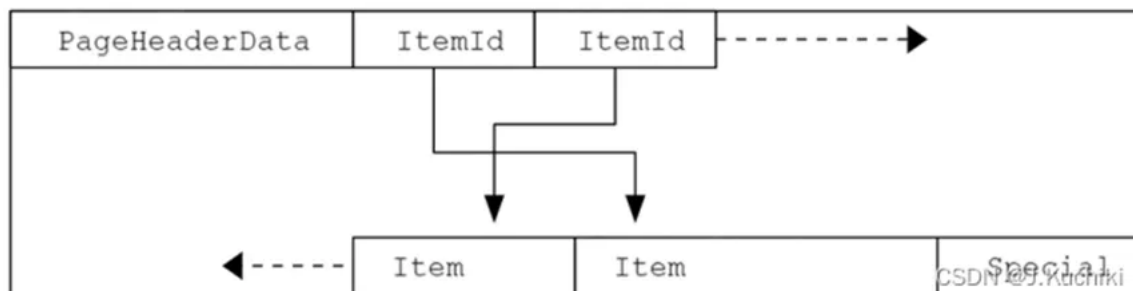
- 6.如果启用了 WAL（写前日志），则记录必要的日志信息。
- 7.在完成所有更改后，结束关键操作区段，释放资源。
- 8.更新缓存失效信息和统计数据。
- 9.返回新插入元组的 OID。

3.1.1.4、RelationPutHeapTuple 函数

RelationPutHeapTuple 函数的主要作用是将一个堆元组放置到指定的缓冲区中的某个页面上。这个过程包括以下几个关键步骤：

- 1、页面获取：首先从缓冲区中获取目标页面的头部。
- 2.事务 ID 设置：为元组数据设置事务 ID。这涉及到判断页面版本，并相应地转换事务 ID。
- 3、元组添加：将元组数据添加到页面中。这通过 PageAddItem 函数完成，它会返回元组在页面上的偏移量编号。如果添加失败，则触发 PANIC。
- 4、位置更新：更新元组的 t_self 字段，以指向元组实际存储的位置。
- 5、CTID 更新：在页面上找到新元组的 ItemId 和 Item，然后更新元组头部的 t_ctid 字段，使其指向实际存储位置。

在 PostgreSQL 中，实际数据被保存到段文件（堆文件）中，并且每个堆文件的大小为 segsize，其大小一般为 1GB（在编译期间可以更改）。为每个段文件设置大小是为了兼容不同平台最大文件的限制。一个段文件包含多个页面块（页面块大小为blocksize，默认为 8KB），页面块的大小不能太小，太小不能存下一个元组，太大则增加了页面读写失败的概率。这里补充一下数据库中的页面布局，如下图所示：



3.1.2、删除元组

3.1.2.1、heap_delete 函数

在 PostgreSQL 中，使用标记删除的方式来删除元组，这对于多版本并发控制（Multi-version Concurrency Control, MVCC）是有好处的，其 Undo 和 Redo 速度是相当高速的，因

为只需重新设置标记即可。被标记删除的磁盘空间会通过运行 VACUUM（清理数据库命令，通常每天运行一次）收回。删除元组主要调用函数 heap_delete 来实现。

heap_delete 函数用于从 PostgreSQL 中的表（即关系）删除一个指定的元组。这个过程包括以下几个关键步骤：

1. 读取和锁定页面：首先读取包含目标元组的页面，并对其进行锁定以进行独占访问。
2. 检查元组状态：检查目标元组的当前状态，确定它是否可以被当前事务删除。
3. 处理并发事务：如果元组被其他事务锁定，可能需要等待并再次检查元组的状态。
4. 执行删除操作：一旦确认可以删除元组，将其在页面上的信息进行更新，标记为删除。
5. 写入 WAL 日志：如果启用了 WAL（写前日志），记录删除操作。
6. 处理 TOAST 数据：如果元组有外部存储（TOAST）数据，也需要相应地处理这些数据。
7. 缓存失效和统计信息更新：更新系统缓存和统计信息以反映删除操作。
8. 释放资源：释放所有占用的资源，包括缓冲区和锁。

3.1.3、更新元组

3.1.3.1、heap_update 函数

元组的更新操作实际上是删除和插入操作的结合，即先标记删除旧元组，再插入新元组。元组的更新由函数 heap_update 实现。

heap_update 函数用于更新表的一个指定元组。这个过程包括以下几个关键步骤：

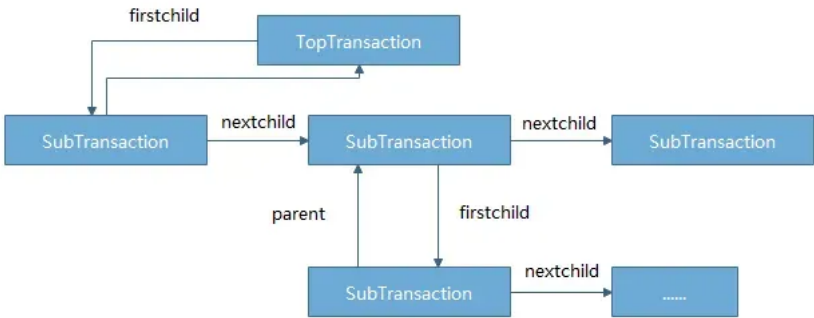
1. 读取和锁定页面：首先读取包含目标元组的页面，并对其进行锁定以进行独占访问。
2. 检查元组状态：检查目标元组的当前状态，确定它是否可以被当前事务更新。
3. 处理并发事务：如果元组被其他事务锁定，可能需要等待并再次检查元组的状态。
4. 执行更新操作：一旦确认可以更新元组，将其在页面上的信息进行更新，标记为更新。
5. 写入 WAL 日志：如果启用了 WAL（写前日志），记录更新操作。
6. 处理 TOAST 数据：如果元组有外部存储（TOAST）数据，也需要相应地处理这些数据。
7. 缓存失效和统计信息更新：更新系统缓存和统计信息以反映更新操作。
8. 释放资源：释放所有占用的资源，包括缓冲区和锁。

值得注意的是，PostgreSQL 中进行删除和更新操作时，被删除或修改的元组并不会从物理文件中删除，而是在事务标记中被标记为无效。因此，当进行过大量的删除和更新操作之后，数据库

数据文件中由于有大量的无效元组，其尺寸会变得异常庞大，此时需要对数据库进行一定的清理操作，这就需要用到 VACUUM 机制。

四、ResourceOwner资源跟踪

PostgreSQL中的ResourceOwner对象用于跟踪每个事务（包括子事务）在执行过程中所占用的内存资源。它简化了对查询相关资源（例如缓冲区锁、表锁等）的管理，确保这些资源在事务结束或失败时能够得到及时释放。



4.1、ResourceOwner结构和功能

与传统的采用牢不可破数据结构的方式相比，ResourceOwner采用了单例模式来跟踪资源。这种方式更加灵活，也更易于实现资源的释放。

ResourceOwner的主要功能包括：

- 记录事务过程中使用到的各种资源，包括：
 - 共享缓冲区：用于存储表数据、索引等数据。
 - 缓冲区pin锁：用于防止缓冲区被替换。
 - Cache：用于存储查询计划、表定义等信息。
 - TupleDesc：用于描述表或列的数据类型。
 - Snapshot：用于表示数据库的某个时间点状态。
- 维护ResourceOwner之间的树状结构，以便于统一管理各个层次的资源。
- 在事务结束或失败时，自动释放所有占用的资源。

ResourceOwner是PostgreSQL中用于跟踪和管理事务资源的对象。它建立在MemoryContext API之上，并提供了更加高级的资源管理功能，包括：

- 能够跟踪和管理各种类型的资源，例如：共享缓冲区、缓存对象、锁等。

- 能够形成对象树，以统一管理各个层次的资源。
- 能够自动释放未使用的资源，提高资源利用率。
- 能够增强系统稳定性，防止内存泄漏。

每个ResourceOwner节点的内存都在TopMemoryContext中进行分配。在内存中保存了三个全局的ResourceOwner结构变量：

- CurrentResourceOwner: 记录当前使用的ResourceOwner
- CurTransactionResourceOwner: 记录当前事务的ResourceOwner
- TopTransactionResourceOwner: 记录顶层事务的ResourceOwner

```
src/include/utils/resowner.h
typedef struct ResourceOwnerData *ResourceOwner;
/* Globally known ResourceOwners */
extern PGDLLIMPORT ResourceOwner CurrentResourceOwner;
extern PGDLLIMPORT ResourceOwner CurTransactionResourceOwner;
extern PGDLLIMPORT ResourceOwner TopTransactionResourceOwner;
extern PGDLLIMPORT ResourceOwner AuxProcessResourceOwner;
src/backend/utils/resowner/resowner.c
typedef struct ResourceOwnerData {
    ResourceOwner parent;      /* NULL if no parent (toplevel owner) */
    ResourceOwner firstchild;  /* head of linked list of children */
    ResourceOwner nextchild;   /* next child of same parent */
    const char *name;          /* name (just for debugging) */
    /* We have built-in support for remembering: */
    ResourceArray bufferarr;    /* owned buffers */
    ResourceArray catrefarr;    /* catcache references */
    ResourceArray catlistrefarr; /* catcache-list pins */
    ResourceArray relrefarr;    /* relcache references */
    ResourceArray planrefarr;   /* plancache references */
    ResourceArray tupdescarr;   /* tupdesc references */
    ResourceArray snapshotarr; /* snapshot references */
    ResourceArray filearr;      /* open temporary files */
    ResourceArray dsmarr;       /* dynamic shm segments */
    ResourceArray jitarr;       /* JIT contexts */
    /* We can remember up to MAX_RESOWNER_LOCKS references to local locks. */
    int nlocks;                 /* number of owned locks */
    LOCALLOCK *locks[MAX_RESOWNER_LOCKS]; /* list of owned locks */
} ResourceOwnerData;
```

每个ResourceOwner的节点都可以拥有子节点，形成对象树。当释放父节点时，所有直接和间接的子节点都将被释放。这种对象树结构使得ResourceOwner能够有效地管理事务的层次结构。

ResourceArray用于存储各种类型的资源ID。它采用了一种混合哈希表和数组的结构来提高存储效率。

4.2、ResourceArray

ResourceArray的内存布局如下：

- itemsarr：存放资源ID的数组。
- invalidval：表示无效资源的特殊值。
- capacity：itemsarr的容量。
- nitems：已存储的资源ID数量。
- maxitems：允许存储的最大资源ID数量。
- lastidx：最后插入或获取的资源ID的索引。

哈希表模式

当资源ID数量较少时，ResourceArray采用哈希表模式存储资源ID。itemsarr数组被划分为多个槽（slot），每个槽可以存储一个资源ID或invalidval。nitems表示有效的资源ID数量。

数组模式

当资源ID数量较多时，ResourceArray采用数组模式存储资源ID。itemsarr数组被直接当作数组使用，nitems表示有效的资源ID数量。

模式切换

如果资源ID数量超过maxitems，ResourceArray 则会将存储模式切换为数组模式。反之，如果资源ID数量低于capacity的一半，则会将存储模式切换为哈希表模式。

性能优化

ResourceArray 采用了一些性能优化措施，包括：

- 使用lastidx记录最后插入或获取的资源ID的索引，以便快速查找资源ID。
- 在哈希表模式下，使用开链哈希表来避免哈希冲突。
- 在数组模式下，预留部分空槽，以便快速插入新资源ID。

如果资源ID过多，我们使用开链哈希表。itemsarr数组是由capacity slot组成的哈希表，每个slot或者存放资源ID或者"invalidval"。nitems是合法的资源ID的数量。如果没有超过maxiems，我们可以增大数组然后重新hash。在这种模式下，maxitems应该小于capacity，这样我们不用花费太多时间搜索空的slot。在这两种模式，lastidex记录的是最后一个插入或者由GetAny返回的位置，这会加快ResourceArrayRemove函数执行。

```
typedef struct ResourceArray {
    Datum      *itemsarr;      /* buffer for storing values */
    Datum      invalidval;     /* value that is considered invalid */
    uint32      capacity;      /* allocated length of itemsarr[] */
    uint32      nitems;        /* how many items are stored in items array */
    uint32      maxitems;      /* current limit on nitems before enlarging */
    uint32      lastidx;       /* index of last item returned by GetAny */
} ResourceArray;
```

PostgreSQL 中的 ResourceOwner 用于跟踪和管理事务和 Portal 所使用的资源。每个事务、子事务和 Portal 都会创建一个 ResourceOwner 对象，用于记录该事务或 Portal 所占用的各种资源，例如共享缓冲区、缓存对象、锁等。

4.3、Portal 的 ResourceOwner

在 Portal 的整个生命周期中，全局变量 CurrentResourceOwner 指向 Portal 的 ResourceOwner 对象。这意味着所有与 Portal 相关的操作，例如 ReadBuffer 和 LockAcquire，都会将所需的资源记录到该 ResourceOwner 中。

Portal 关闭时的处理

当 Portal 关闭时，任何未释放的资源（通常是锁）都会成为当前事务的责任。这通过将 Portal 的 ResourceOwner 变为当前事务 ResourceOwner 的子对象来实现。在 resowner.c 中，当释放子对象时，资源会自动传输给父对象。

子事务的 ResourceOwner

子事务的 ResourceOwner 也是其直接父对象的子对象。这意味着子事务所使用的所有资源都会被记录在其父事务的 ResourceOwner 中。

事务和 Portal ResourceOwner 的必要性

PostgreSQL 需要事务和 Portal 相关的 ResourceOwner，因为事务可能在尚未有 Portal 关联时进行要求资源的初始化工作，例如查询分析

为每个事务创建一个 ResourceOwner

```

static void AtStart_ResourceOwner(void) {
    TransactionState s = CurrentTransactionState;

    /* We shouldn't have a transaction resource owner already. */
    Assert(TopTransactionResourceOwner == NULL);
    /* Create a toplevel resource owner for the transaction. */
    s->curTransactionOwner = ResourceOwnerCreate(NULL, "TopTransaction");

    TopTransactionResourceOwner = s->curTransactionOwner;
    CurTransactionResourceOwner = s->curTransactionOwner;
    CurrentResourceOwner = s->curTransactionOwner;
}

```

为每个子事务创建一个ResourceOwner

```

static void AtSubStart_ResourceOwner(void) {
    TransactionState s = CurrentTransactionState;
    Assert(s->parent != NULL);
    /* Create a resource owner for the subtransaction. We make it a child of the immediate parent's resource owner. */
    s->curTransactionOwner = ResourceOwnerCreate(s->parent->curTransactionOwner, "SubTransaction");

    CurTransactionResourceOwner = s->curTransactionOwner;
    CurrentResourceOwner = s->curTransactionOwner;
}

```

为每个Portal创建一个ResourceOwner


```

/* CreatePortal Returns a new portal given a name.
 * allowDup: if true, automatically drop any pre-existing portal of the same name (if false, an error is raised).
 * dupSilent: if true, don't even emit a WARNING. */
Portal CreatePortal(const char *name, bool allowDup, bool dupSilent) {
    Portal portal;
    AssertArg(PointerIsValid(name));
    portal = GetPortalByName(name);
    if (PortalIsValid(portal)) {
        if (!allowDup)
            ereport(ERROR, (errcode(ERRCODE_DUPLICATE_CURSOR), errmsg("cursor \"%s\" already exists", name)));
        if (!dupSilent)
            ereport(WARNING, (errcode(ERRCODE_DUPLICATE_CURSOR), errmsg("closing existing cursor \"%s\"", name)));
        PortalDrop(portal, false);
    }
    /* make new portal structure */
    portal = (Portal) MemoryContextAllocZero(TopPortalContext, sizeof *portal);
    /* initialize portal context; typically it won't store much */
    portal->portalContext = AllocSetContextCreate(TopPortalContext, "PortalContext", ALLOCSET_SMALL_SIZES);

    /* create a resource owner for the portal */
    portal->resowner = ResourceOwnerCreate(CurTransactionResourceOwner, "Portal");

    /* initialize portal fields that don't start off zero */
    portal->status = PORTAL_NEW;
    portal->cleanup = PortalCleanup;
    portal->createSubid = GetCurrentSubTransactionId();
    portal->activeSubid = portal->createSubid;
    portal->strategy = PORTAL_MULTI_QUERY;
    portal->cursorOptions = CURSOR_OPT_NO_SCROLL;
    portal->atStart = true;
    portal->atEnd = true; /* disallow fetches until query is set */
    portal->visible = true;
    portal->creation_time = GetCurrentStatementStartTimestamp();
    /* put portal in table (sets portal->name) */
    PortalHashTableInsert(portal, name);
    /* reuse portal->name copy */
    MemoryContextSetIdentifier(portal->portalContext, portal->name);
    return portal;
}

```

ResourceOwner 是 PostgreSQL 中用于跟踪和管理事务和 Portal 所使用的资源的重要数据结构。它提供了一套基本API，用于创建、管理和释放 ResourceOwner 对象及其关联的资源。

4.4、基本API概况

1. 创建 ResourceOwner:

- ResourceOwnerCreate(): 创建一个新的 ResourceOwner 对象。
- 参数: 无
- 返回值: 创建成功的 ResourceOwner 对象指针

2. 资源关联和解除关联:

- ResourceOwnerRegister(): 将指定的资源注册到指定的ResourceOwner 对象中。
- 参数: ResourceOwner 对象指针, 资源指针
- 返回值: 无
- ResourceOwnerUnregister(): 将指定的资源从指定的 ResourceOwner 对象中注销。

- 参数：ResourceOwner 对象指针，资源指针
- 返回值：无

3. 释放 ResourceOwner 的拥有资源:

- ResourceOwnerRelease(): 释放所有由指定的 ResourceOwner 对象管理的资源。
- 参数：ResourceOwner 对象指针
- 返回值：无

4. 删除ResourceOwner 对象:

- ResourceOwnerDestroy(): 销毁指定的 ResourceOwner 对象及其所有子对象。
- 参数：ResourceOwner 对象指针
- 返回值：无

特殊处理：锁

由于锁的特殊性，在释放 ResourceOwner 时需要特殊处理。在非错误情况下，无论是在子事务中还是 Portal 申请的锁，锁的持有会直到事务结束。因此，当isCommit为true时，在子对象（child ResourceOwner）中的释放操作只会将锁的所有权（lock ownership）传递给父对象，而不是真的释放掉锁。

目前支持的资源类型

目前，ResourceOwner 直接支持以下资源类型的属主关系维护：

- 缓冲区 pin
- LGMgr 锁
- Catcache
- Relcache
- Tupdesc 引用

其他类型的对象可以通过记录所属 ResourceOwner 的指针与 ResourceOwner 进行关联。

释放时扫描数据

在其他模块中，存在这样一个API，当 ResourceOwner 释放时会得到控制权（get control），来扫描自己的数据以发现需要被删除的对象。

CurrentResourceOwner 的使用

当 unpin 缓冲区、释放锁或 cache 引用时，当前的CurrentResourceOwner必须指向它们申请时相同的 ResourceOwner。通过额外的记录可以放松这个限制，但目前来看没有这个必要。代

码中，若有CurrentResourceOwner的暂时变化，应使用PG_TRY结构，以确保当出现错误退出时，前一CurrentResourceOwner能正确恢复。

五、实验部分

5.1、FULL VACUUM实验

创建表

```
postgres=# CREATE TABLE large_table (  
postgres(#      id BIGSERIAL PRIMARY KEY,  
postgres(#      data TEXT  
postgres(# );  
CREATE TABLE
```

插入数据

```
postgres=# INSERT INTO large_table (data)  
postgres=# SELECT repeat('This is some sample data. ', 100)  
postgres=# FROM generate_series(1, 5000000);  
  
INSERT 0 5000000
```

修改数据

```
postgres=# UPDATE large_table  
postgres=# SET data = 'This is some different sample data. '  
postgres=# ;  
UPDATE 5000000
```

执行全局清除

```
postgres=# VACUUM (FULL, VERBOSE) large_table;  
INFO:  vacuuming "public.large_table"  
INFO:  "public.large_table": found 5000000 removable, 5000000 nonremovable row versions in 111402 pages  
DETAIL:  0 dead row versions cannot be removed yet.  
CPU: user: 1.08 s, system: 0.67 s, elapsed: 2.42 s.  
VACUUM
```

由于对表进行了修改，所以可以找到5百万个可删除的版本，并将其进行删除。由于未产生事务，所以未产生死亡的版本。死亡版本的产生途径可以是事务中被修改的版本，也可以是被冻结的版本。所以也可以进行手动冻结，产生死亡版本。

5.2、扫描表

创建示例表

```

postgres=# CREATE TABLE demotable (num numeric, id int);
CREATE TABLE
postgres=# CREATE INDEX demoidx ON demotable(num);
CREATE INDEX
postgres=# INSERT INTO demotable SELECT random() * 1000, generate_series(1, 1000000);
INSERT 0 1000000
postgres=# analyze;
ANALYZE

```

Sequential Scan (顺序扫描)：顺序扫描就是顺序扫描对应表所有页的item指针。如果一个表有100页，每页有1000条记录，顺序扫描就会获取100*1000条记录并检查是满足可见性规则以及过滤条件。因此，即使只有1条记录满足条件，他也会扫描100K条记录。针对上表的数据，下面的查询会进行顺序扫描，因为需要选择大部分的数据。

```

postgres=# explain SELECT * FROM demotable WHERE num < 21000;
               QUERY PLAN
-----
Seq Scan on demotable  (cost=0.00..17990.00 rows=1000000 width=15)
  Filter: (num < '21000'::numeric)
(2 rows)

```

Index Scan (索引扫描)：根据查询中涉及到的索引，使用不同的数据结构(根据索引类型)，并且根据谓词使用非常小的扫描量定位所需数据。然后，使用索引扫描找到的条目直接指向堆区域中的数据(如上图所示)，然后根据可见性规则，获取数据。

索引扫描有两个步骤：

1.从索引相关的数据结构中获取数据。它返回表中相应数据的TID。

2.然后直接访问相应的表页以获得整个数据。需要这一额外步骤的原因如下：--查询可能请求获取比相应索引列更多列。--可见性信息不与索引数据一起维护。因此，为了按照隔离级别检查数据的可见性，它需要访问表数据。

注意：因为索引扫描涉及的成本与我们所做的I/O类型有关，一般只有在总体增益超过随机I/O开销时，才选择索引扫描。

```

postgres=# explain SELECT * FROM demotable WHERE num = 21000;
               QUERY PLAN
-----
Index Scan using demoidx on demotable  (cost=0.42..8.44 rows=1 width=15)
  Index Cond: (num = '21000'::numeric)
(2 rows)

```

Index Only Scan (仅索引扫描)：类似于索引扫描，但它只扫描索引数据结构。与索引扫描相比，选择仅索引扫描还有两个附加的先决条件：

1、查询要获取的列属于索引一部分。

2、对应heap页面上的所有元组都应该可见。如上所述，索引数据结构不维护可见性信息，因此为了仅从索引中获取数据，我们应该避免检查可见性，如果该页面的所有数据都被视为可见，则可能会发生这种情况。

以下查询将使用仅进行索引扫描。尽管此查询和上面的查询几乎相同，但由于只选择num字段，因此它将选择仅索引扫描。

```
postgres=# explain SELECT num FROM demotable WHERE num = 21000;
               QUERY PLAN
-----
Index Only Scan using demoidx on demotable (cost=0.42..4.44 rows=1 width=11)
  Index Cond: (num = '21000'::numeric)
(2 rows)
```

Bitmap Scan（位图扫描）：是索引扫描和顺序扫描的合体。它试图解决索引扫描的缺点(随机读)，但仍然保持其全部优点。如上所述，对于在索引数据结构中找到的每个数据，它需要在堆页面中找到相应的数据。因此，它需要先获取一次索引页，然后再获取堆页，这会导致大量随机I/O。位图扫描方法利用了索引扫描的优势，而无需随机I/O。工作流程分为以下两个阶段：

1、Bitmap Index Scan:首先，它从索引数据结构中获取所有数据，并创建所有TID的位图。位图分为有损位图和无损位图。如果是有损位图的话，位图中记录的是整个页面，所以需要recheck。

2、Bitmap Heap Scan:它读取位图，然后从与页面和偏移量对应的堆中扫描数据。最后，它检查可见性和谓词等，并根据所有这些检查的结果返回元组。

以下查询将使用位图扫描，因为它没有选择很少的记录(比索引扫描多),同时也没有选择大量的记录(比顺序扫描少)。

```
postgres=# explain SELECT * FROM demotable WHERE num < 210;
               QUERY PLAN
-----
Bitmap Heap Scan on demotable (cost=5574.86..13636.54 rows=205734 width=15)
  Recheck Cond: (num < '210'::numeric)
    -> Bitmap Index Scan on demoidx (cost=0.00..5523.43 rows=205734 width=0)
          Index Cond: (num < '210'::numeric)
(4 rows)
```

TID Scan（TID扫描）：是PostgreSQL中一种非常特殊的扫描，只有当查询谓词中有TID时才会选择tid scan。以下为TID扫描的查询：

```

postgres=# select ctid from demotable where id=21000;
      ctid
-----
(115,57)
(1 row)

postgres=# explain select * from demotable where ctid='(115,57)';
              QUERY PLAN
-----
Tid Scan on demotable  (cost=0.00..4.01 rows=1 width=15)
   TID Cond: (ctid = '(115,57)::tid)
(2 rows)

```

参考链接: <https://zhuanlan.zhihu.com/p/658078161>

5.3、TOAST

现在我们来实际验证下TOAST:

```

1 postgres=# insert into blog values(1, 'title', '0123456789');INSERT 0 1postgres=# select
2 * from blog;
3 id | title | content ----+-----+----- 1 | title | 0123456789(1 row)
4 postgres=# select * from pg_toast.pg_toast_16441;
5 chunk_id | chunk_seq | chunk_data -----+-----+----- (0 rows)

```

可以看到因为content 只有10 个字符, 所以没有压缩, 也没有行外存储。然后我们使用如下SQL 语句增加content 的长度, 每次增长1 倍, 同时观察content 的长度, 看看会发生什么情况?

```

1 postgres=# update blog set content=content||content where id=1;UPDATE 1postgres=# select
2 id,title,length(content) from blog;
3 id | title | length ----+-----+----- 1 | title | 20(1 row)postgres=# select
4 * from pg_toast.pg_toast_16441;
5 chunk_id | chunk_seq | chunk_data -----+-----+----- (0 rows)

```

反复执行如上过程, 直到pg_toast_16441 表中有数据:

```

1 postgres=# select id,title,length(content) from blog;
2 id | title | length ----+-----+----- 1 | title | 327680(1 row)
3 postgres=# select chunk_id,chunk_seq,length(chunk_data) from pg_toast.pg_toast_16441;
4 chunk_id | chunk_seq | length -----+-----+----- 16439 | 0 |
5 1996
6 16439 | 1 | 1773(2 rows)

```

可以看到，直到content 的长度为327680 时（已远远超过页大小8K），对应TOAST 表中才有了2行数据，且长度都是略小于2K，这是因为extended 策略下，先启用了压缩，然后才使用行外存储。

下面我们将content 的TOAST 策略改为EXTERNAL，以禁止压缩。

```

1 postgres=# alter table blog alter content set storage external;ALTER TABLE postgres=#
2 \d+ blog;
3
4          Table "public.blog"
5 Column | Type | Modifiers | Storage | Stats target | Description
6 -----+-----+-----+-----+-----+-----
7 id
8 |
9 integer | | plain | |
10 title | text | | extended | |
11 content | text | | external | |

```

然后我们再插入一条数据：

```

1 postgres=# insert into blog values(2, 'title', '0123456789');INSERT 0 1postgres=# select
2 id,title,length(content) from blog;
3 id | title | length ----+-----+----- 1 | title | 327680
4 2 | title | 10(2 rows)

```

然后重复以上步骤，直到pg_toast_16441 表中有数据：

```

1 postgres=# update blog set content=content||content where id=2;UPDATE 1pos
gres=# select
2 id,title,length(content) from blog;
3 id | title | length ----+-----+----- 2 | title | 327680
4 1 | title | 327680(2 rows)
5 postgres=# select chunk_id,chunk_seq,length(chunk_data) from pg_toast.pg_t
oast_16441;
6 chunk_id | chunk_seq | length -----+-----+----- 16447 | 0 |
7 1996
8 16447 | 1 | 1773
9 16448 | 0 | 1996
10 16448 | 1 | 1996
11 16448 | 2 | 1996
12 .... (省略)
13 16448 | 164 | 1996(167 rows)

```

因为不允许压缩，所以新的操作在TOAST 表中生成了更多Chunk 块行记录。通过以上操作得出以下结论：

- 如果策略允许压缩，则TOAST 优先选择压缩。
- 不管是否压缩，一旦数据超过2KB 左右，就会启用行外存储。
- 修改TOAST 策略，不会影响现有数据的存储方式。

5.4、系统表

<https://developer.aliyun.com/article/990160>